

АННОТАЦИЯ

Пояснительная записка к курсовому проекту посвящена разработке программы на языке С для управления базой данных об учёных с использованием линейных двунаправленных списков. Целью работы является практическое закрепление навыков работы с динамическими структурами данных, а именно создание, обработка и хранение списков в оперативной памяти. В проекте реализованы основные операции: добавление, удаление, редактирование, поиск и сортировка записей по различным полям (ID, ФИО, учёная степень, область науки, количество статей, цитирований, индекс Хирша). Программа поддерживает импорт и экспорт данных в текстовые и бинарные файлы, а также выводит по пять лучших учёных в каждой научной области по индексу Хирша и количеству цитирований. Для удобства просмотра таблицы реализован постраничный скроллинг с управлением клавишами. Работа демонстрирует эффективность применения линейных списков для организации динамических баз данных без ограничения на количество записей.

ОГЛАВЛЕНИЕ

АННОТАЦИЯ.....	2
ВВЕДЕНИЕ.....	5
2 НАЗНАЧЕНИЕ И ОБЛАСТЬ ПРИМЕНЕНИЯ ПРОГРАММЫ	7
3 ТЕХНИЧЕСКИЕ ХАРАКТЕРИСТИКИ ПРОГРАММЫ.....	8
3.1 Постановка задачи на разработку программы	8
3.2 Применяемые математические методы.....	8
3.3 Описание и обоснование выбора метода организации входных, выходных и промежуточных данных.....	9
3.4 Обоснование выбора языка и среды программирования	9
3.5 Разработка модульной структуры программы.....	10
3.6 Описание алгоритмов функционирования программы.....	12
3.7 Обоснование состава технических и программных средств	26
4 ВЫПОЛНЕНИЕ ПРОГРАММЫ	27
4.1 Условия выполнения программы.....	27
4.2 Загрузка и запуск программы.....	28
4.3 Общий вид главного меню.....	28
4.4 Описание работы с программой.....	29
4.4.1 Пункт 1 – «Организация списка»	29
4.4.2 Пункт 2 – «Просмотр таблицы»	30
4.4.3 Пункт 3 – «Добавление новой записи»	31
4.4.4 Пункт 4 – «Удаление записи»	32
4.4.5 Пункт 5 – «Корректировка записи».....	33
4.4.6 Пункт 6 – «Сортировка данных».....	35
4.4.7 Пункт 7 – «Поиск записи».....	37
4.4.8 Пункт 8 – «Сохранить таблицу в файл»	38
4.4.9 Пункт 9 – «Чтение таблицы из файла».....	40
4.4.10 Пункт 10 – «Вывести по 5 учёных с наибольшим индексом Хирша и кол-вом цитирований для каждой области».....	41
4.4.11 Пункт 11 – «Выход из программы».....	42

5	ЗАКЛЮЧЕНИЕ	43
6	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	45
	ПРИЛОЖЕНИЕ А.....	46

ВВЕДЕНИЕ

Наименование программы – «Программа для работы с базой сведений об учёных с использованием линейных двунаправленных списков». Разработка ведётся на основании технического задания, утверждённого кафедрой «Информационные технологии и системы» Севастопольского государственного университета 7 марта 2026 года (вариант задания №4).

Актуальность задачи обусловлена необходимостью обработки и хранения структурированных данных об учёных (ФИО, научная область, учёная степень, наукометрические показатели) без жёсткого ограничения на количество записей. Использование линейных списков позволяет динамически изменять размер базы данных, эффективно выполнять операции вставки, удаления, поиска и сортировки. Такие структуры широко применяются в учебных и прикладных системах, где объём данных заранее неизвестен.

Целью курсовой работы является разработка программы на языке C, которая обеспечивает ведение базы данных об учёных на основе двунаправленного линейного списка с возможностью добавления, редактирования, удаления, сортировки, поиска, импорта/экспорта записей, а также вывода топ-5 учёных по каждой научной области.

Для достижения поставленной цели необходимо решить следующие задачи:

- Разработать структуры данных для хранения информации об учёном и элемента списка.
- Реализовать функции создания списка, добавления в начало и конец, удаления одного или всех элементов.
- Создать алгоритмы линейного поиска и сортировки (пузырьковым методом) по всем полям записи.
- Реализовать страничный вывод таблицы на экран с управлением клавишами.
- Обеспечить запись/чтение данных в текстовый и бинарный файлы.
- Реализовать специальную функцию вывода топ-5 учёных по индексу Хирша и по количеству цитирований для каждой области науки.

В пояснительной записке далее рассматриваются: назначение и область применения программы, её технические характеристики (постановка задачи, математические методы, выбор языка и среды, модульная структура, алгоритмы, технические средства), описание выполнения программы (условия запуска, работа с меню, экранные формы), заключение и список использованных источников.

2 НАЗНАЧЕНИЕ И ОБЛАСТЬ ПРИМЕНЕНИЯ ПРОГРАММЫ

Программа предназначена для автоматизации работы с небольшими и средними базами данных об учёных. Она позволяет вводить, корректировать, удалять, сортировать и искать записи, а также сохранять их в файлы и восстанавливать из файлов.

Область применения:

- Учебные лабораторные работы по дисциплинам «Программирование», «Структуры и алгоритмы обработки данных».
- Подготовка отчётов по наукометрическим показателям для кафедр или научных подразделений.
- Демонстрация принципов работы динамических структур данных (двунаправленные списки) в консольных приложениях на языке С.

Программа может быть использована как основа для более сложных информационных систем с графическим интерфейсом или интеграцией с внешними базами данных.

3 ТЕХНИЧЕСКИЕ ХАРАКТЕРИСТИКИ ПРОГРАММЫ

3.1 Постановка задачи на разработку программы

В соответствии с вариантом задания №4 (утверждён 07.03.2026) необходимо разработать программу, которая:

- Хранит записи об учёных в оперативной памяти в виде двунаправленного линейного списка.
- Каждая запись содержит: уникальный целочисленный идентификатор (ID), ФИО (до 75 символов), научную область (до 25 символов), учёную степень (до 25 символов), количество статей (целое беззнаковое), количество цитирований (целое беззнаковое), индекс Хирша (целое беззнаковое).
- Поддерживает функции: вывод всей таблицы с постраничной навигацией, добавление записи (в начало или конец), удаление записи по ID или всех записей, редактирование любого поля, сортировку по любому полю (пузырьковым методом), поиск записей по значению любого поля, экспорт в текстовый (.txt) и бинарный (.bin) файлы, импорт из таких же файлов.
- Дополнительная функция: вывести для каждой научной области топ-5 учёных по индексу Хирша и топ-5 по количеству цитирований, результат сохранить в файл 0results-success.txt.

3.2 Применяемые математические методы

В программе не используются сложные математические расчёты. Основные алгоритмические решения:

- Сортировка – метод «пузырька» с обменом содержимого структур. Сравнение элементов выполняется по заданному полю с помощью функции `compareScientist`, возвращающей разность для числовых полей или результат `strcmp` для строковых. Временная сложность в худшем случае – $O(n^2)$, что приемлемо для учебных целей и небольшого объёма данных.

- Поиск – линейный последовательный перебор элементов списка. Сложность $O(n)$.
- Генерация ID – используется глобальный счётчик nextID, который пересчитывается как $\max(ID) + 1$ при импорте или удалении, что гарантирует уникальность.

3.3 Описание и обоснование выбора метода организации входных, выходных и промежуточных данных

Входные данные:

- С клавиатуры: значения полей записи об учёном (строки и целые числа).
- Из текстового файла: каждая строка содержит поля, разделённые символом табуляции: ID, ФИО, Учёная степень, Область науки, Статьи, Цитирования, Индекс Хирша.
- Из бинарного файла: массив структур `struct scientist` без разделителей.

Выходные данные:

- Вывод на экран таблицы учёных, введённых ранее, а также различные системные сообщения (например, сообщения об ошибках или сообщения об успешных операциях над базой учёных).
- Вывод (сохранение) в текстовый файл, каждая строка которого содержит поля, разделённые символом табуляции: ID, ФИО, Учёная степень, Область науки, Статьи, Цитирования, Индекс Хирша.
- Вывод (сохранение) в бинарный файл в виде массива структур `struct scientist` без разделителей.

3.4 Обоснование выбора языка и среды программирования

- Язык C выбран в соответствии с заданием курсовой работы (линейные списки в языке C/C++). Основные преимущества:
- Прямая работа с указателями и динамическим выделением памяти.

- Высокая производительность.
- Стандартные библиотеки `stdio.h`, `stdlib.h`, `string.h` предоставляют все необходимые функции для файлового ввода-вывода и обработки строк.
- Кроссплатформенность (код может быть скомпилирован в Linux, но для Windows-специфичной функции `SetConsoleOutputCP` требуется условная компиляция или замена).

Среда программирования – Visual Studio Code версии 1.123.0 с компилятором MinGW (или любым другим, поддерживающим стандарт C99). Выбор обусловлен удобством редактирования и наличием отладчика.

3.5 Разработка модульной структуры программы

Декомпозиция задачи выполнена по функциональному признаку. Выделены следующие модули (функции):

Таблица 3.1 – Функции и их назначение

Наименование модуля (функции)	Назначение
main	Организация главного цикла, интерактивное меню с управлением стрелками, обработка выбора пунктов 1–11, вызов соответствующих функций.
readData	Ввод одной записи с клавиатуры с проверкой числовых полей через <code>isNumber</code> .
addFirst, addLast	Добавление узла в начало или конец списка.
createListFromKeyboard	Многократный вызов <code>addFirst/addLast</code> для создания списка посредством чтения данных с клавиатуры.
viewList	Постраничный вывод таблицы с использованием клавиш-стрелок и ESC.
printTableHeader, printNode	Вспомогательные функции вывода заголовка и одной строки.
editElement	Поиск узла по ID и изменение выбранного поля.

deleteNode	Удаление элемента по ID или всех элементов; возврат количества удалённых.
sortTable	Пузырьковая сортировка обменом данных между узлами (поля info).
compareScientist	Сравнение двух структур по заданному полю.
findScientist	Линейный поиск и вывод всех записей, удовлетворяющих критерию.
exportToTxt, exportToBin	Сохранение списка в текстовый или бинарный файл.
importFromTxt, importFromBin	Чтение списка из текстового или бинарного файла.
checkFileExt	Добавление расширения .txt или .bin, если его нет.
writeToTextFile	Запись одного узла в текстовый файл (используется в экспорте и top5ByArea).
top5ByArea	Формирование для каждой уникальной области науки временного списка, его сортировка по индексу Хирша и по цитированиям, вывод первых 5 в файл.
removeDuplicatesFromFile	Удаление дубликатов строк в результирующем файле (по ID).
setNewID	Пересчёт глобального счётчика nextID после загрузки или удаления.
charToInt, isNumber	Преобразование строки в целое число с проверкой корректности.
captcha	Генерация арифметической капчи (сложение/вычитание чисел от 1 до 50) для подтверждения критических действий.

Главная функция `main` управляет меню и вызывает остальные функции в зависимости от выбора пользователя. Указатель на начало списка `head` передаётся по значению, но функции, изменяющие структуру (`addFirst`, `addLast`, `deleteNode`, `editElement`, `importFromTxt`, `importFromBin`), возвращают обновлённый `head`. Критические действия (создание нового списка, удаление всех записей, импорт, выход без сохранения) предваряются вызовом `captcha`. Вспомогательные функции (`printTableHeader`, `printNode`, `checkFileExt`, `setNewID`) используются внутри основных модулей.

3.6 Описание алгоритмов функционирования программы

В данном подразделе приводится описание алгоритмов основных модулей программы в виде схем программ(модулей), приведённых на рисунках 3.1–3.17.

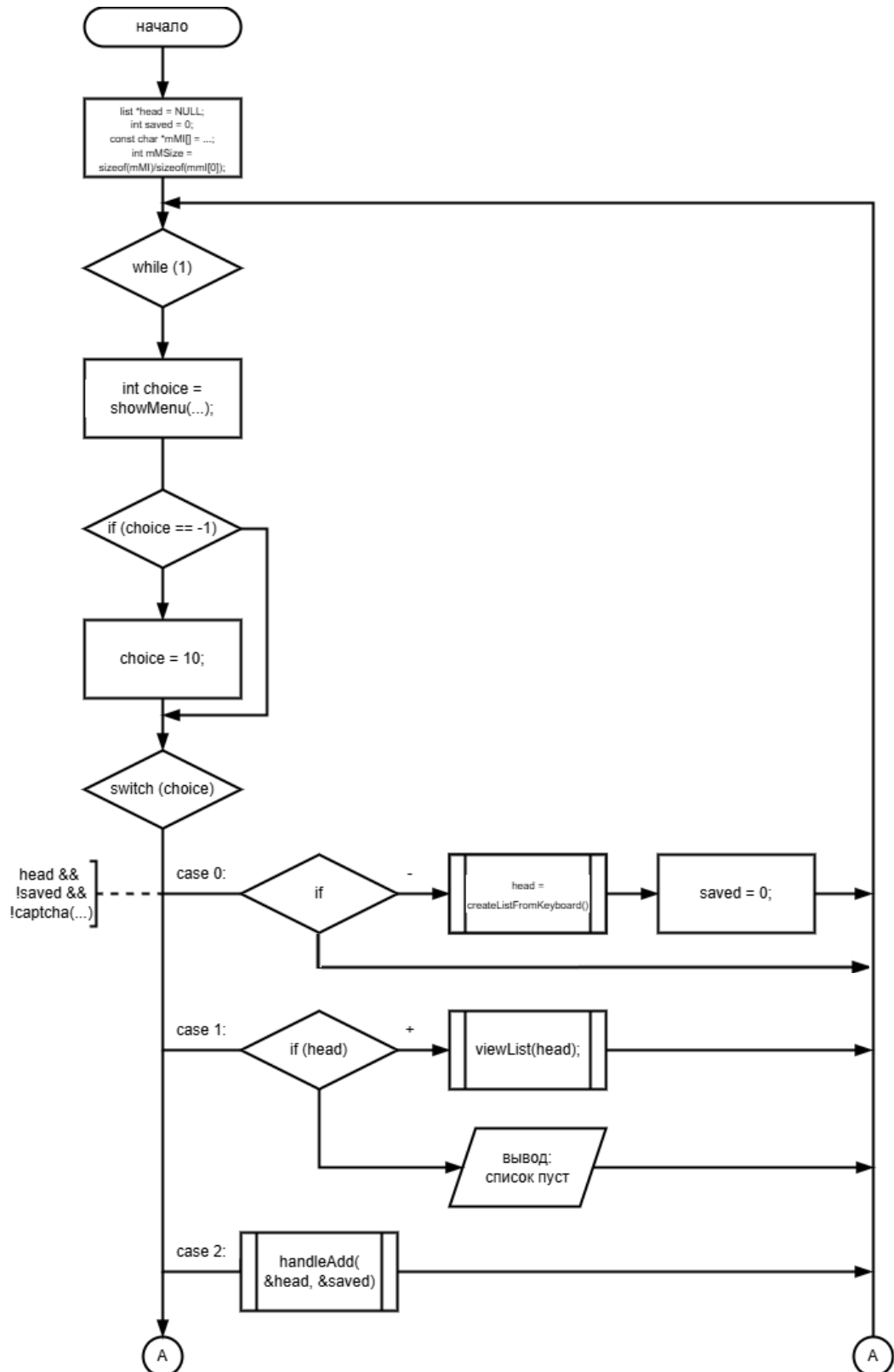
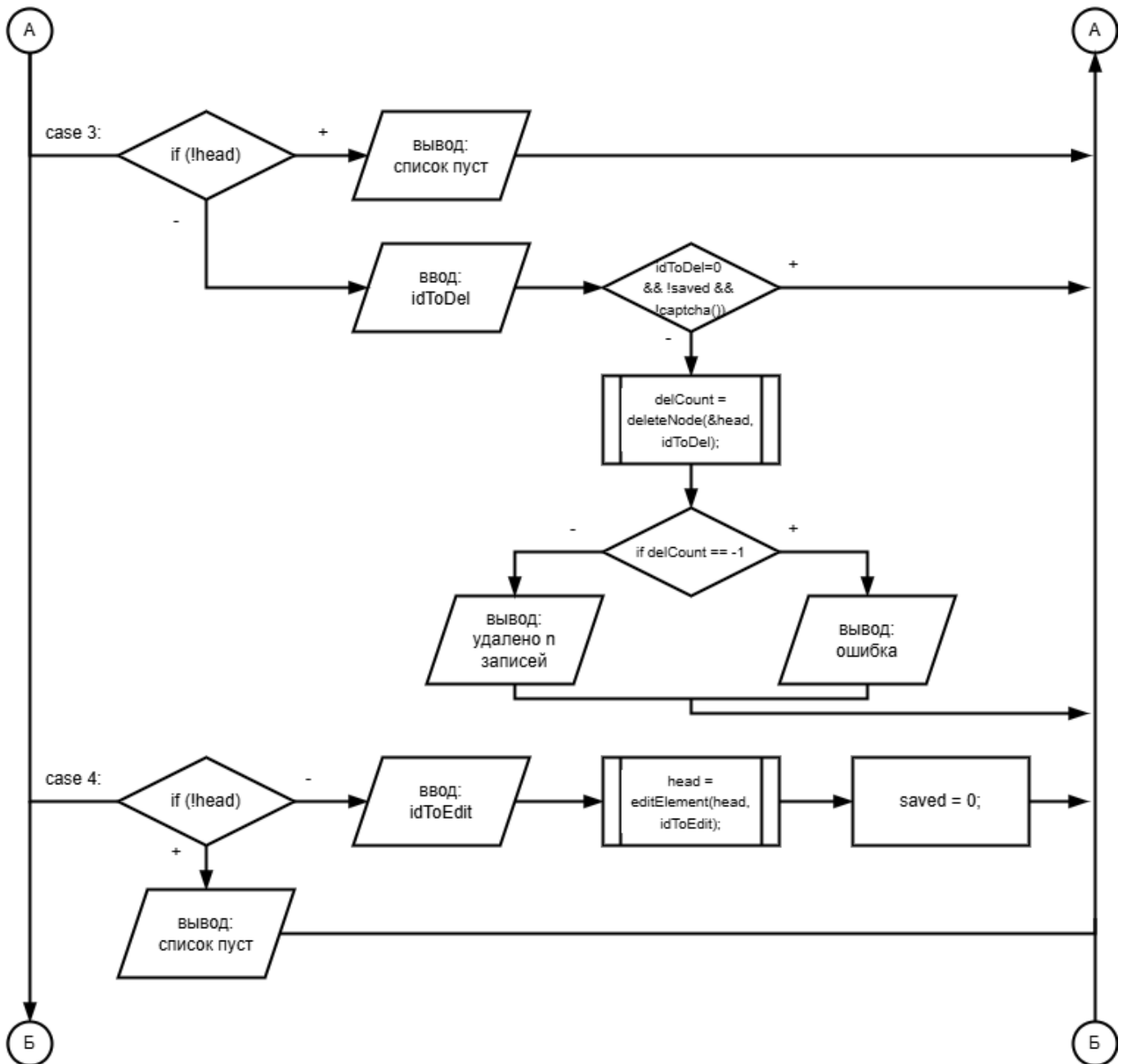


Рисунок 3.1 – Структурная схема функции main (часть 1)

Рисунок 3.2 – Структурная схема функции `main` (часть 2)

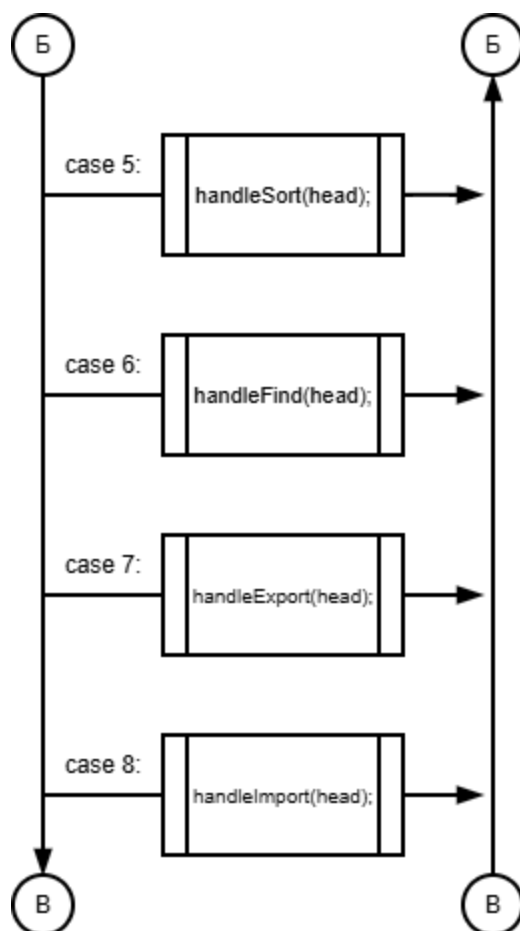


Рисунок 3.3 – Структурная схема функции main (часть 3)

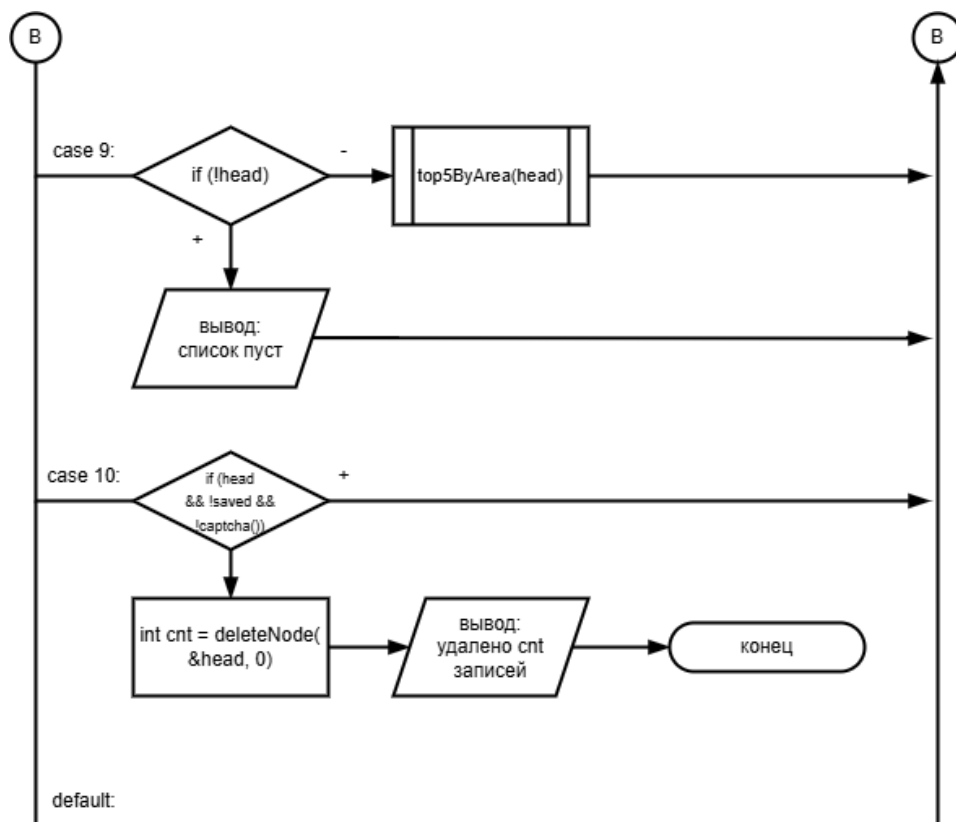


Рисунок 3.4 – Структурная схема функции main (часть 4)

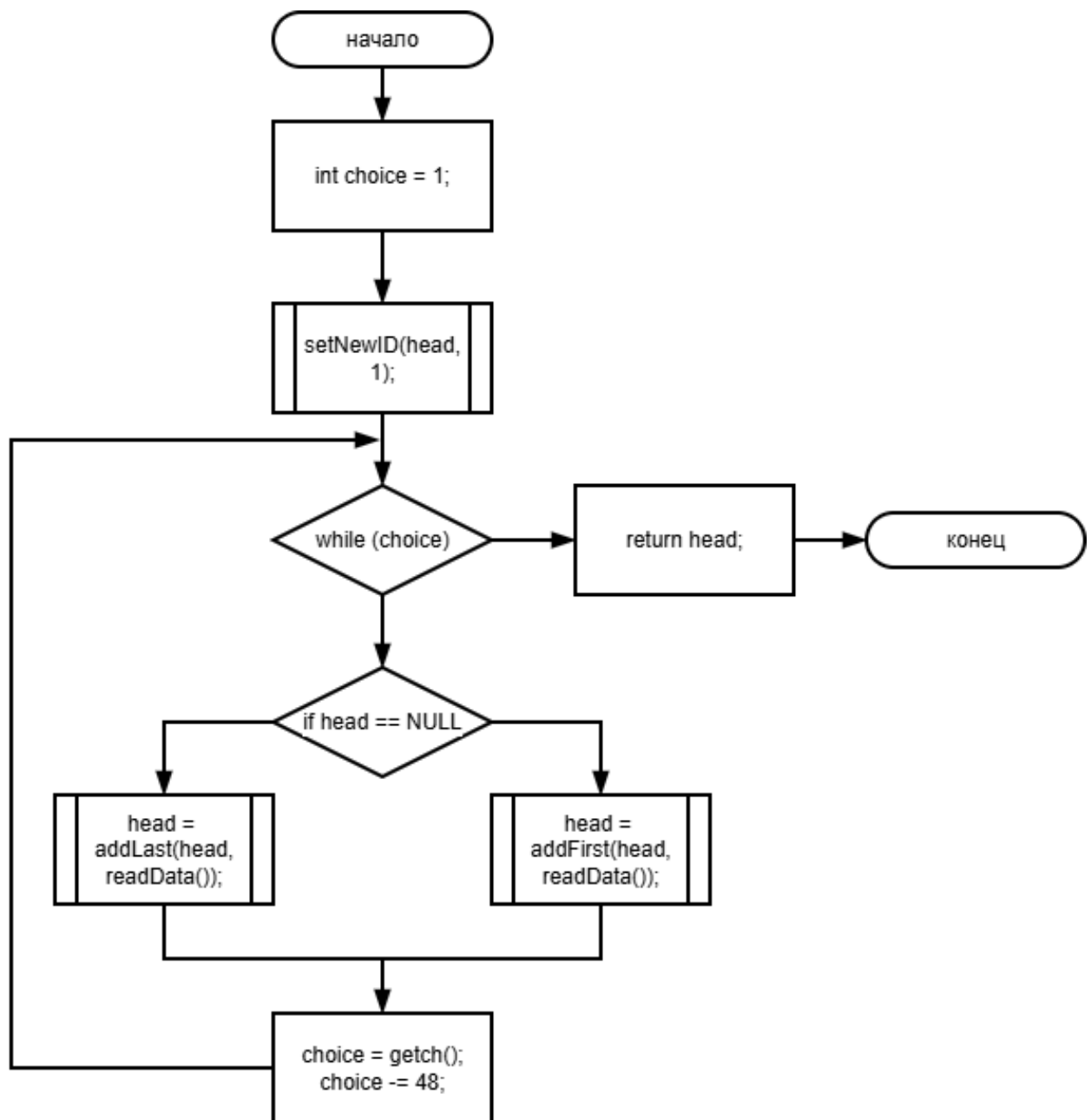


Рисунок 3.5 – Структурная схема функции `createListFromKeyboard`

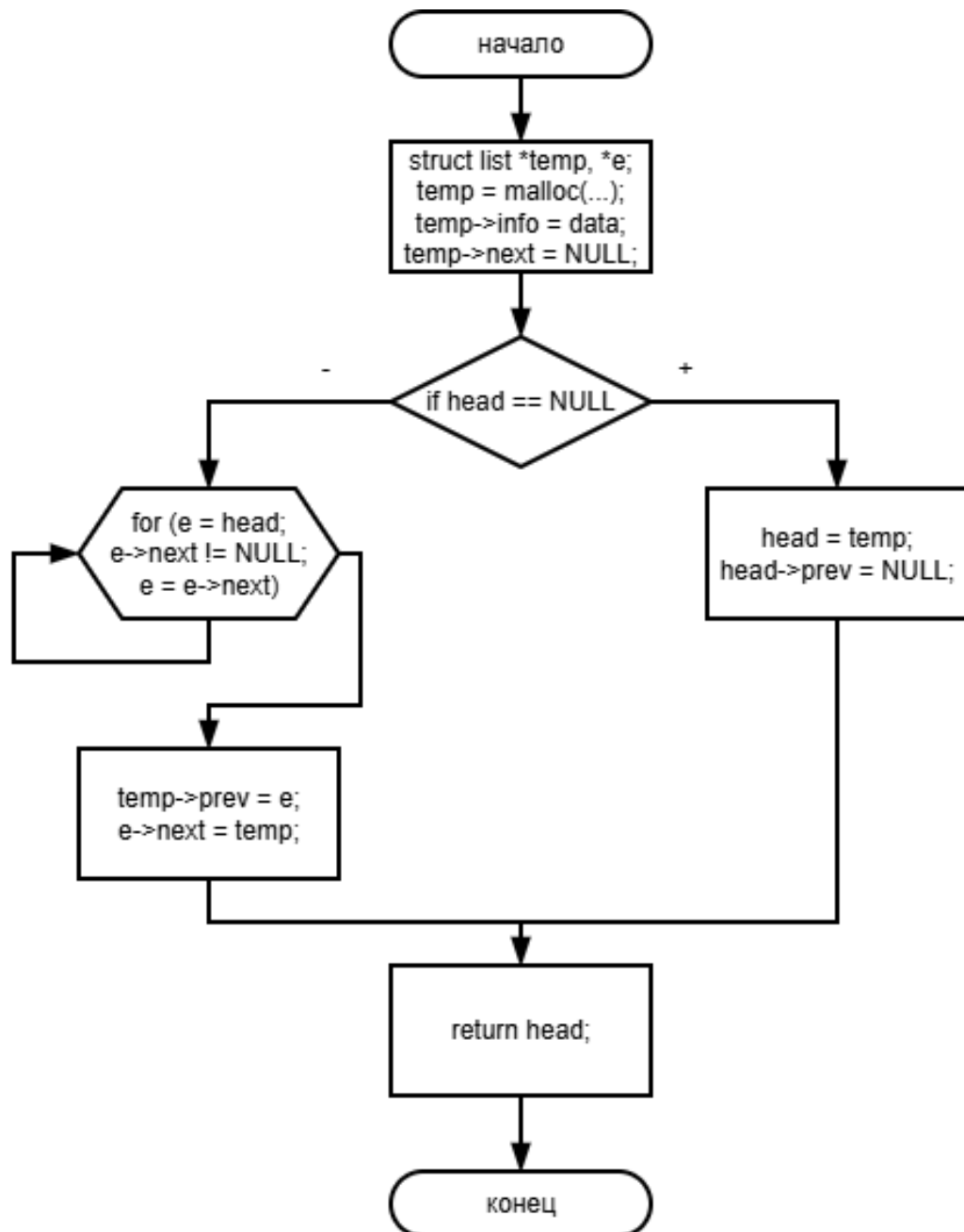


Рисунок 3.6 – Структурная схема функции addLast

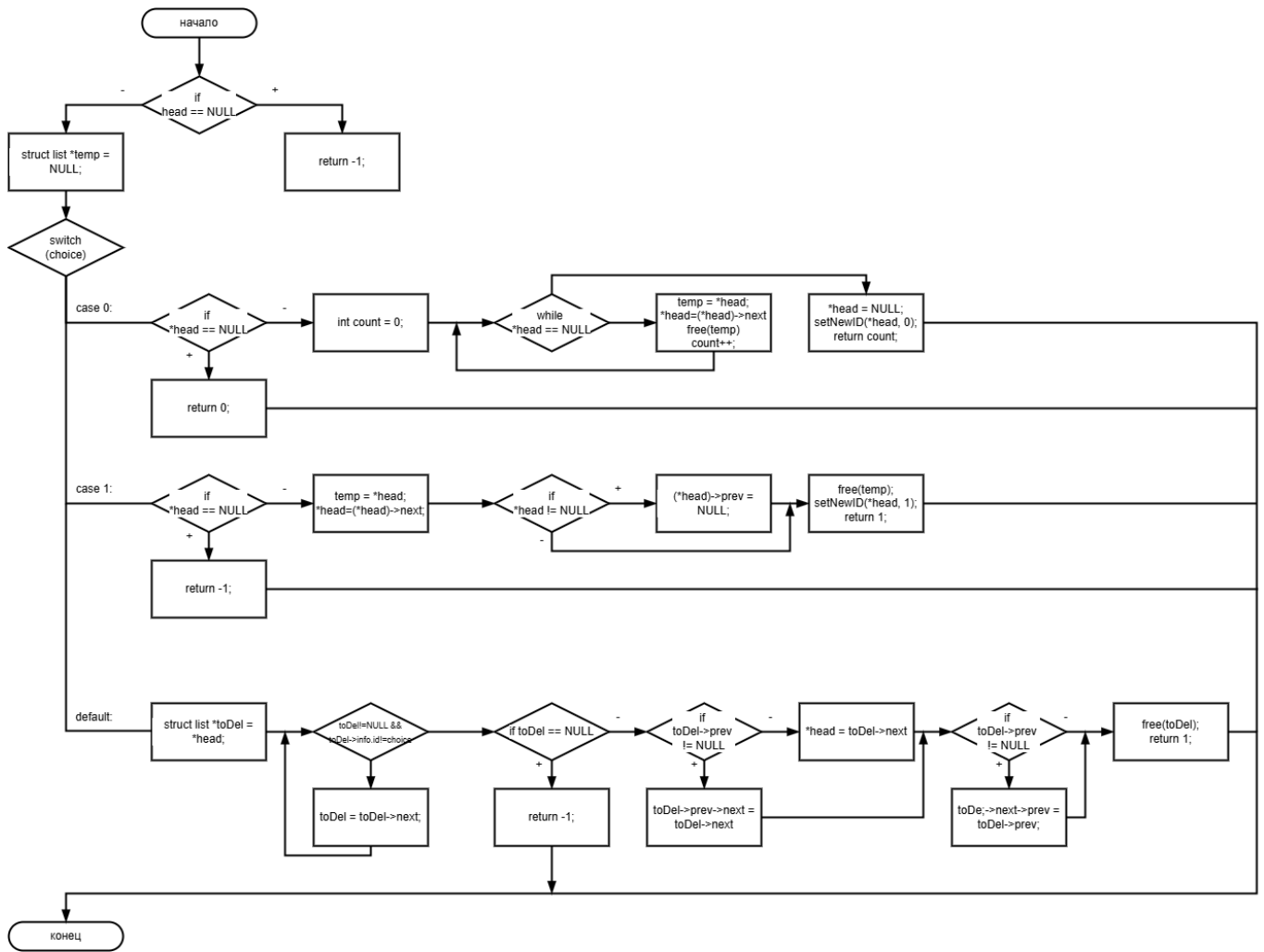


Рисунок 3.7 – Структурная схема функции deleteNode

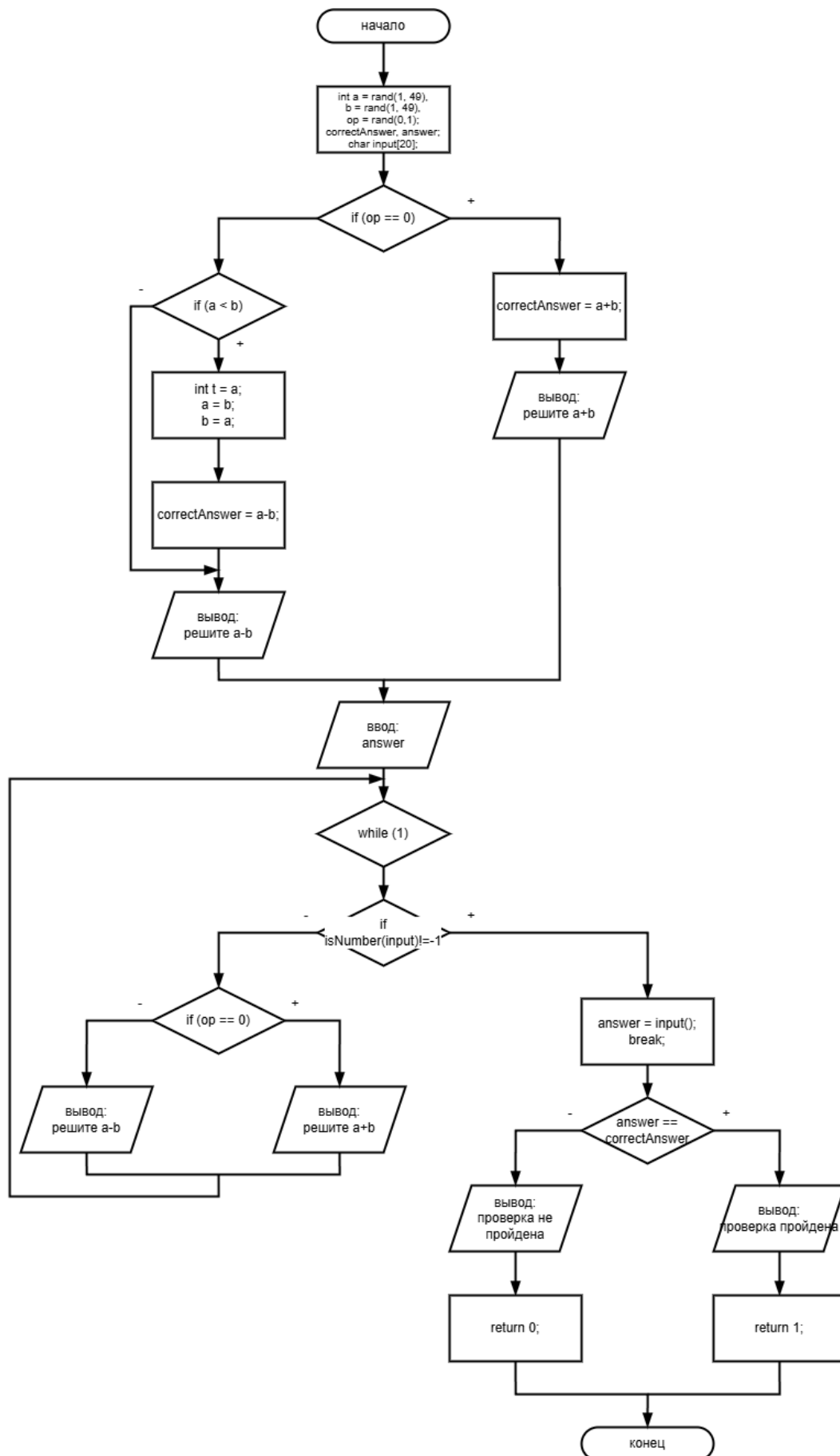


Рисунок 3.8 – Структурная схема функции captcha

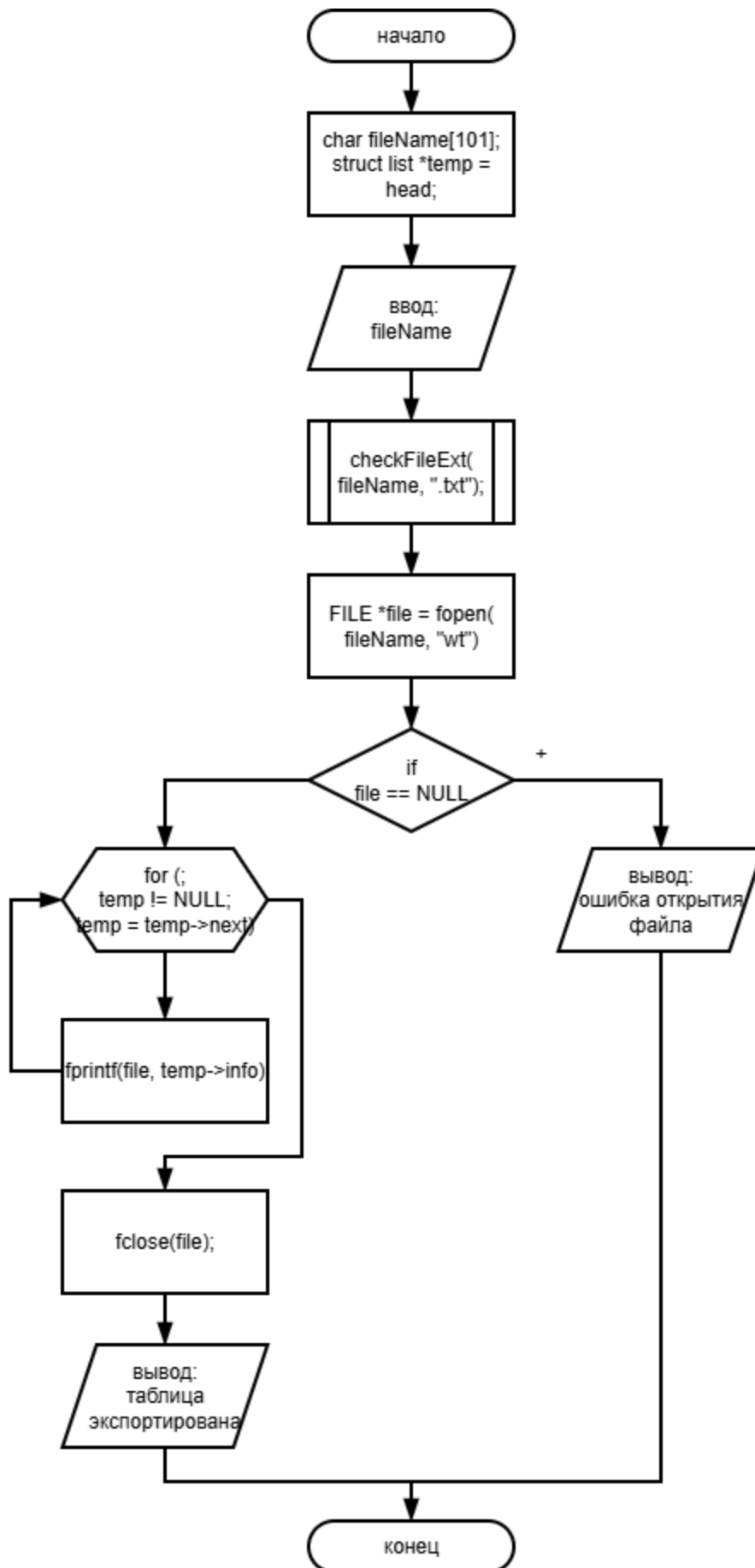


Рисунок 3.9 – Структурная схема функции exportToTxt

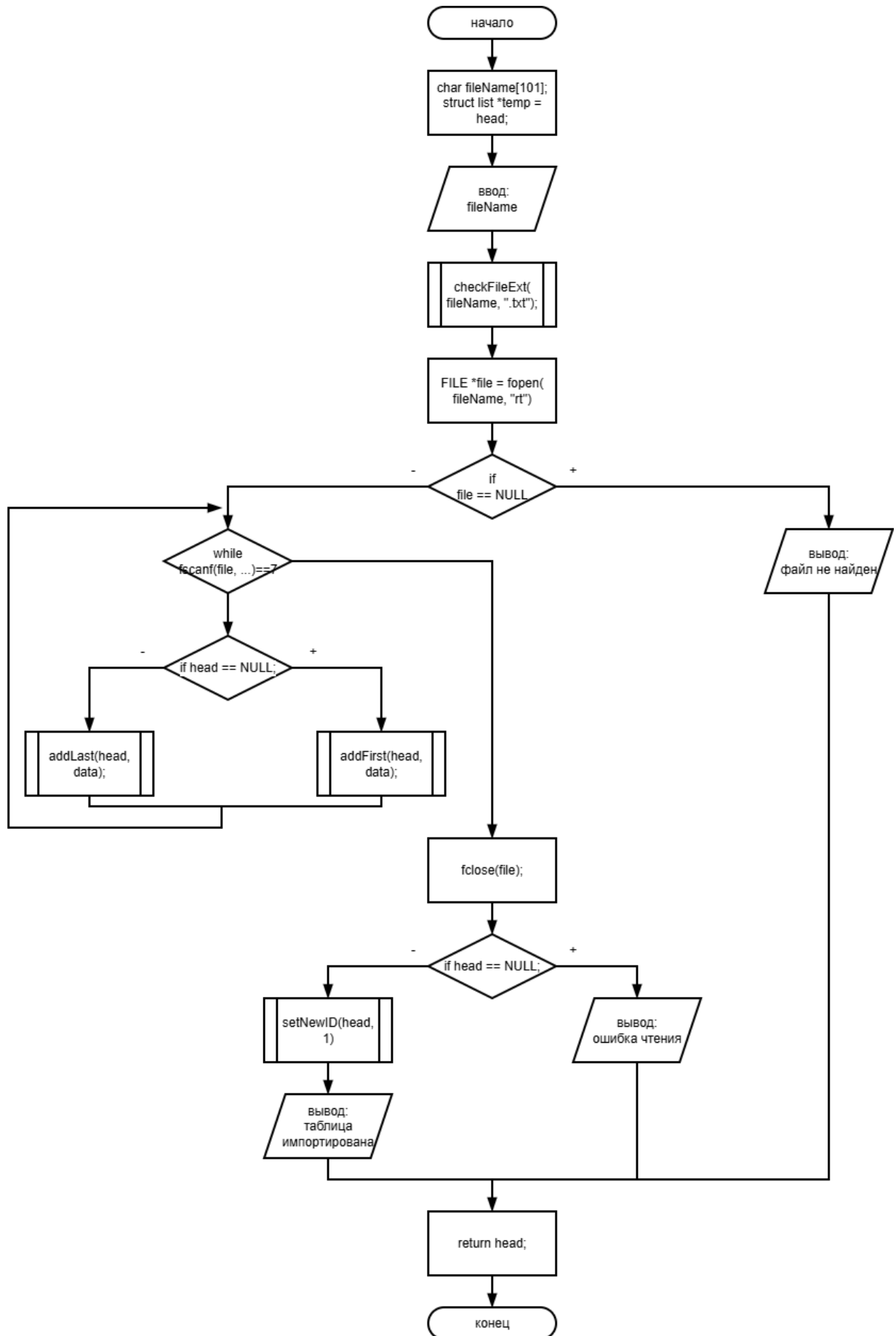


Рисунок 3.10 – Структурная схема функции importFromTxt

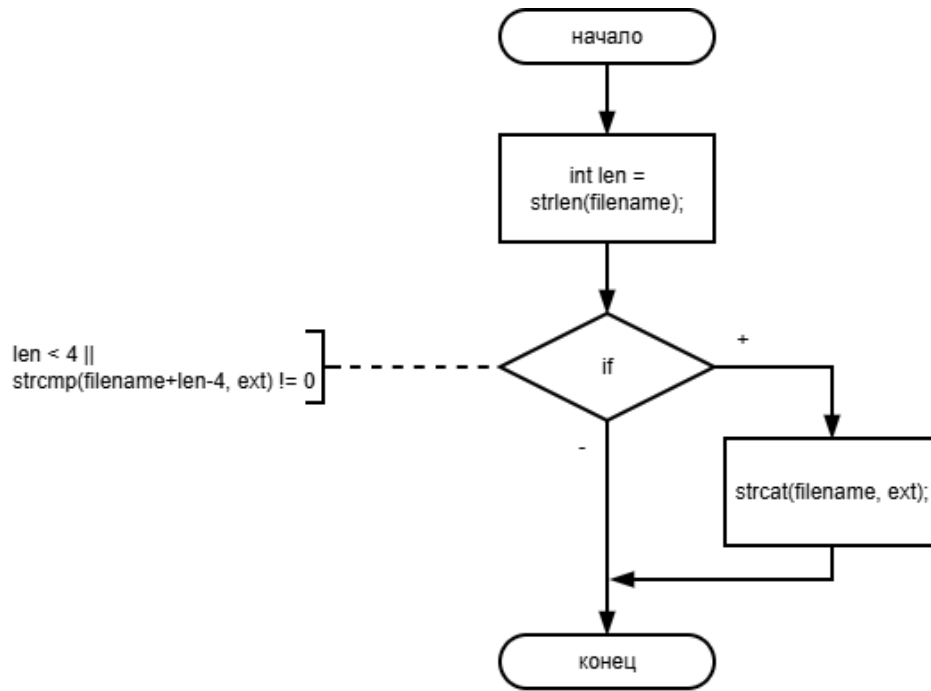


Рисунок 3.11 – Структурная схема функции checkFileExt

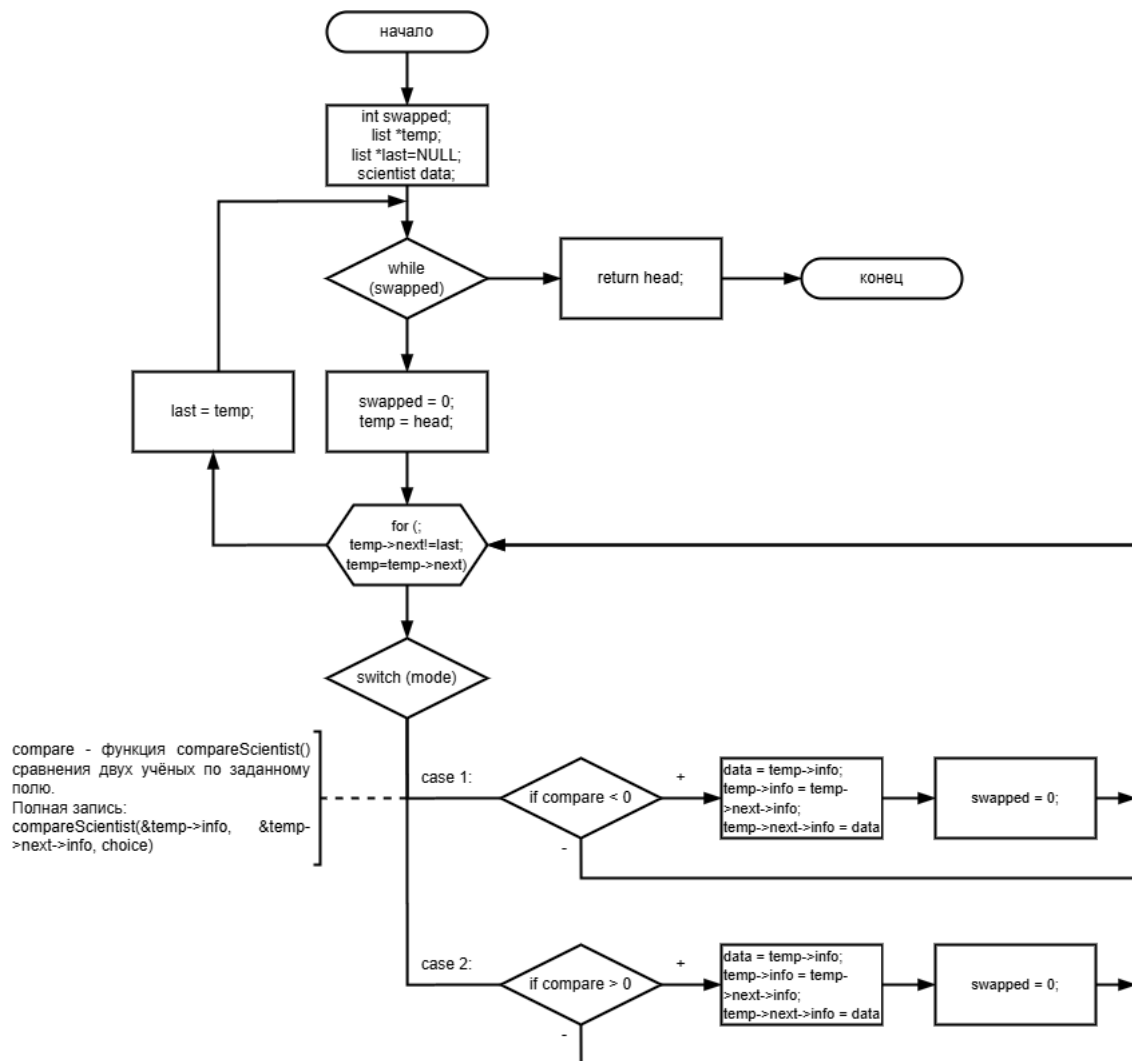


Рисунок 3.12 – Структурная схема функции sortTable

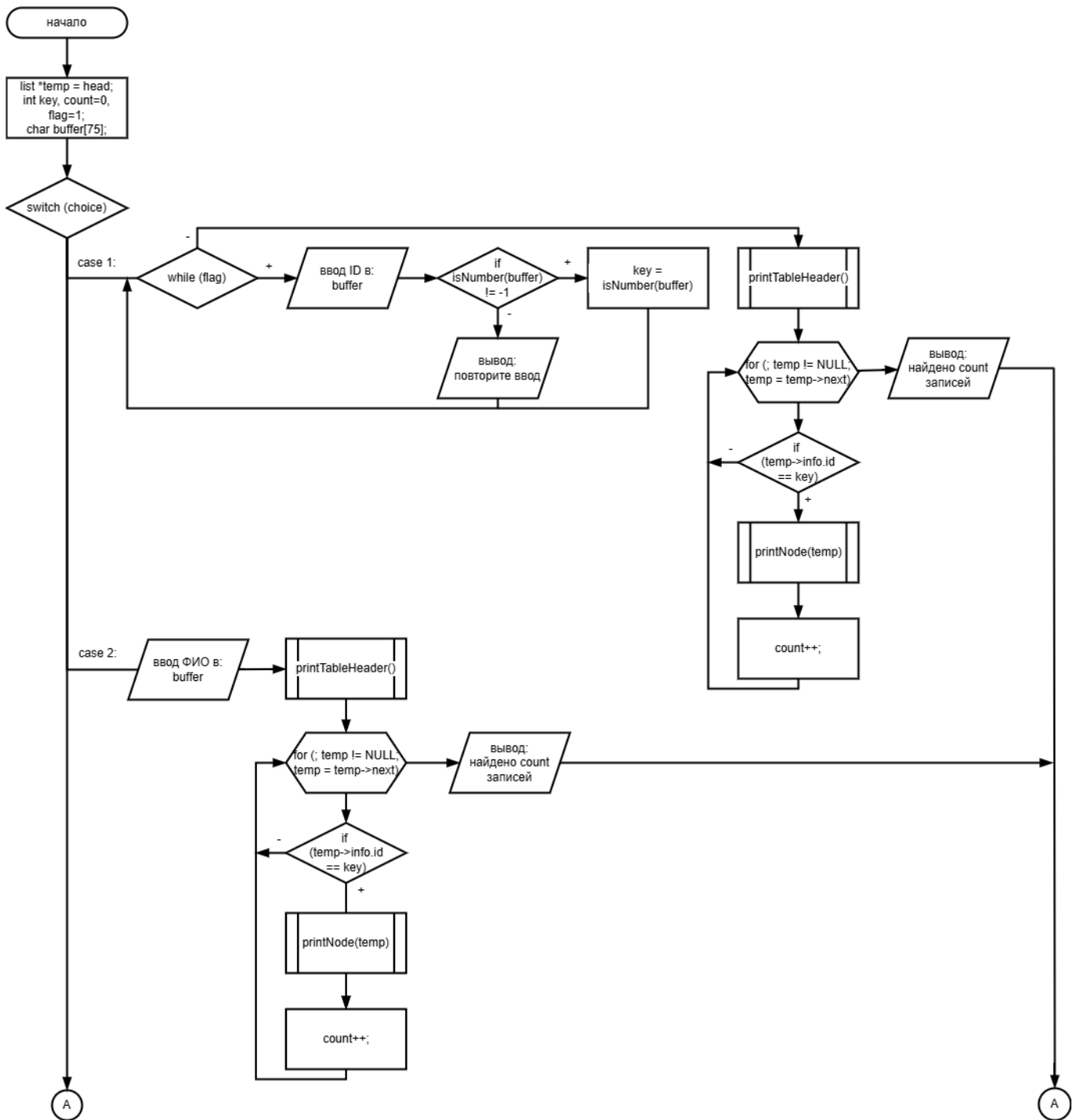
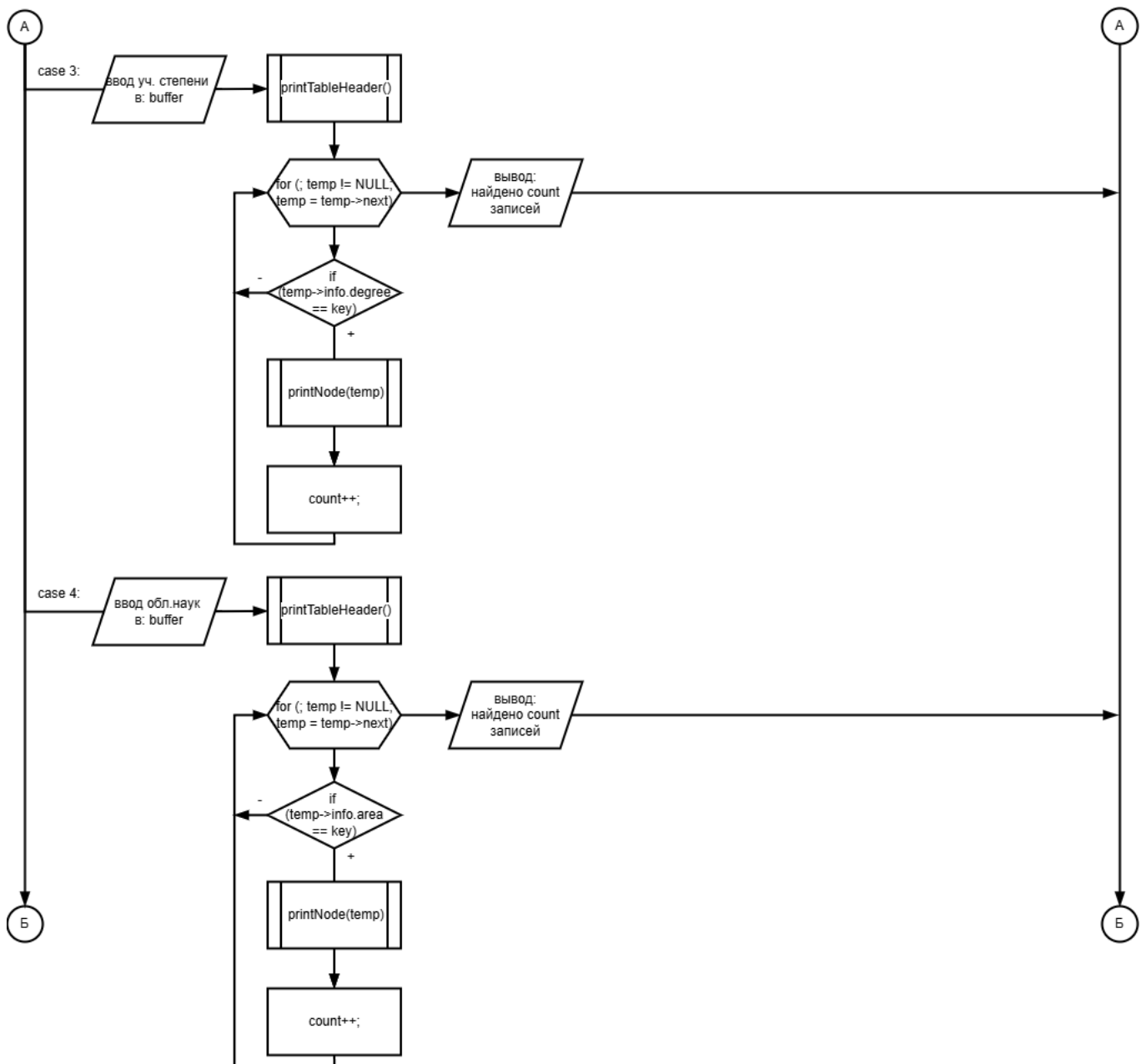


Рисунок 3.13 – Структурная схема функции findScientist (часть 1)

Рисунок 3.14 – Структурная схема функции `findScientist` (часть 2)

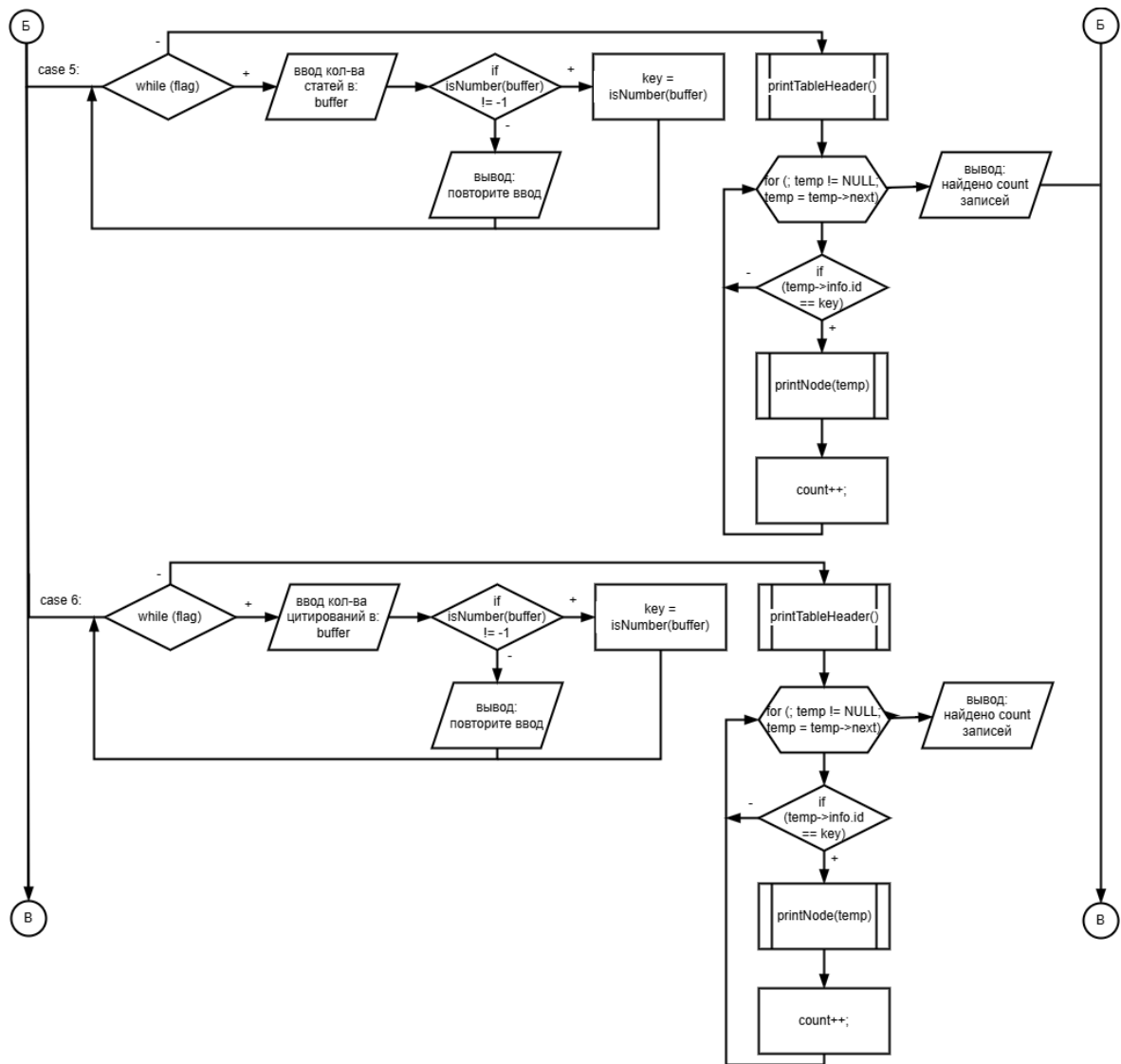


Рисунок 3.15 – Структурная схема функции findScientist (часть 3)

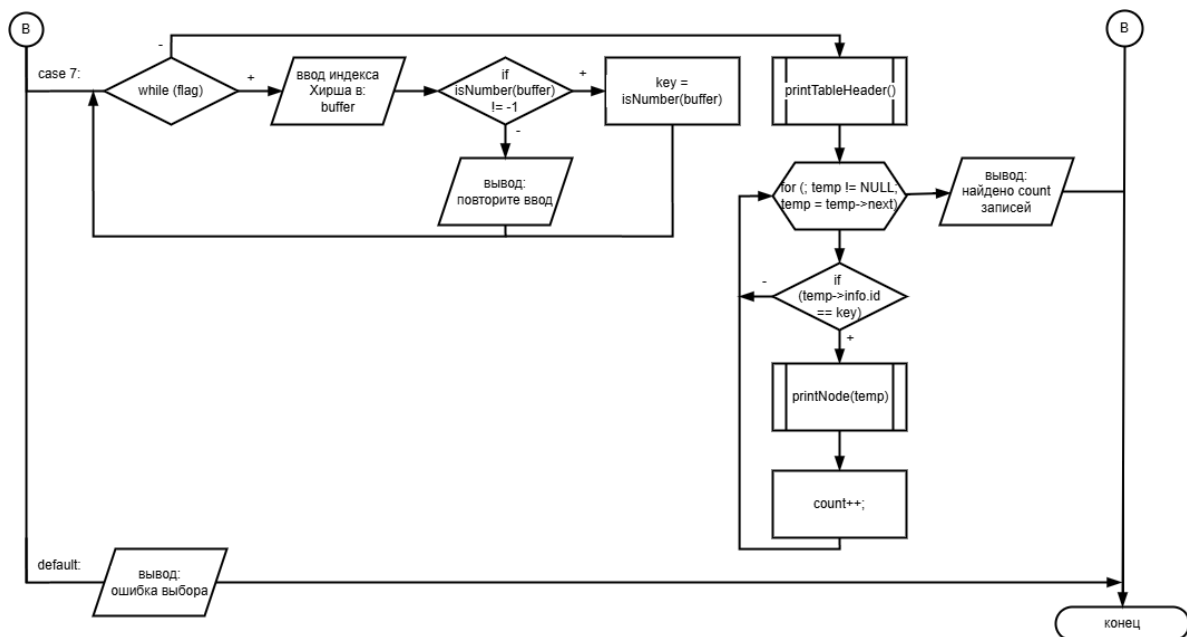


Рисунок 3.16 – Структурная схема функции findScientist (часть 4)

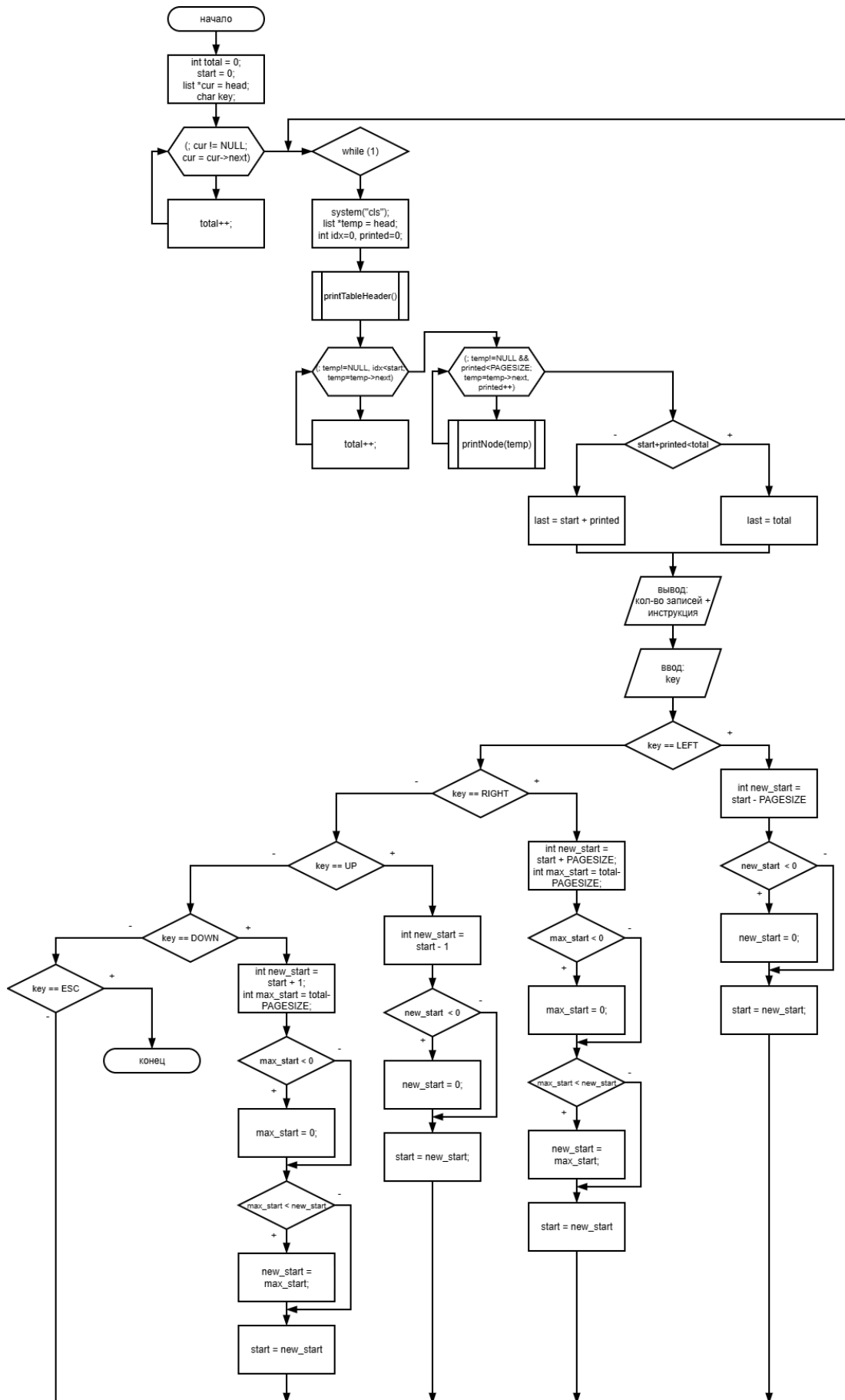


Рисунок 3.17 – Структурная схема функции viewList

Рисунок 3.18 – Структурная схема функции main

3.7 Обоснование состава технических и программных средств

Технические средства:

- Процессор с тактовой частотой не менее 1 ГГц.
- Оперативная память – не менее 256 МБ (фактическое потребление определяется количеством записей: каждая структура `struct scientist` занимает примерно $76+26+26 = 128$ байт + накладные расходы на указатели ($2*8=16$ байт) ≈ 150 байт на узел. Для 10 000 записей потребуется $\sim 1,5$ МБ, что пренебрежимо мало).
- Жёсткий диск (для хранения файлов).
- Клавиатура, монитор.

Программные средства:

- Операционная система семейства Windows (7/10/11).
- Компилятор, поддерживающий стандарт C99 (например, GCC, MinGW, MSVC).
- Среда разработки Visual Studio Code (или любой текстовый редактор).
- Объём оперативной памяти, требуемый для кода программы, не превышает 200 КБ (скомпилированный исполняемый файл). Динамически выделяемая память для списка рассчитывается по формуле: количество_записей * `sizeof(struct list)`. При максимальном разумном количестве записей (например, 10 000) потребуется около $150 \times 10000 \approx 1,5$ МБ, что допустимо для любых современных компьютеров.

4 ВЫПОЛНЕНИЕ ПРОГРАММЫ

4.1 Условия выполнения программы

Для корректной работы программы необходимо обеспечить соблюдение минимальных требований к аппаратному и программному обеспечению.

Таблица 4.1 – Аппаратные требования

Компонент	Минимальные характеристики
Процессор	Intel Pentium 4 / AMD Athlon 64 или совместимый, тактовая частота 1,5 ГГц
Оперативная память	256 МБ (рекомендуется 512 МБ)
Свободное место на жёстком диске	20 МБ для исполняемого файла и вспомогательных файлов
Устройства ввода	Клавиатура (обязательно наличие клавиш со стрелками, Enter, Esc)
Монитор	Поддержка текстового режима 80×25 символов

Таблица 4.2 – Программные требования

Компонент	Требование
Операционная система	Microsoft Windows 7 / 8 / 10 / 11 (32-бит или 64-бит)
Поддержка кодировки	Консоль с поддержкой UTF-8 (стандартная консоль Windows)
Дополнительное ПО	Не требуется (программа поставляется в виде скомпилированного .exe)

Примечания по совместимости:

- Программа использует функции `SetConsoleOutputCP(65001)` и `SetConsoleCP(65001)` для корректного отображения кириллицы. В некоторых

версиях Windows может потребоваться установка шрифта консоли, поддерживающего кириллицу (Lucida Console, Consolas).

- Управление осуществляется через клавиатуру; использование мыши не предусмотрено.
- Для работы с файлами программа требует прав на чтение и запись в текущем каталоге.

4.2 Загрузка и запуск программы

Программа распространяется в виде одного исполняемого файла с расширением .exe. Инсталляция не требует специальных действий – достаточно скопировать файл в любой каталог на жёстком диске.

Порядок инсталляции:

- Создать папку, например C:\ScientistsBase
- Переместить в неё файл scientist.exe
- При необходимости создать ярлык на рабочем столе для быстрого запуска

Запуск программы:

- Дважды щёлкнуть левой кнопкой мыши по файлу scientist.exe (или выбрать «Открыть» в контекстном меню).
- Откроется окно консоли Windows с главным меню программы.

4.3 Общий вид главного меню

После запуска на экране отображается главное меню, изображённое на Рис. 4.1

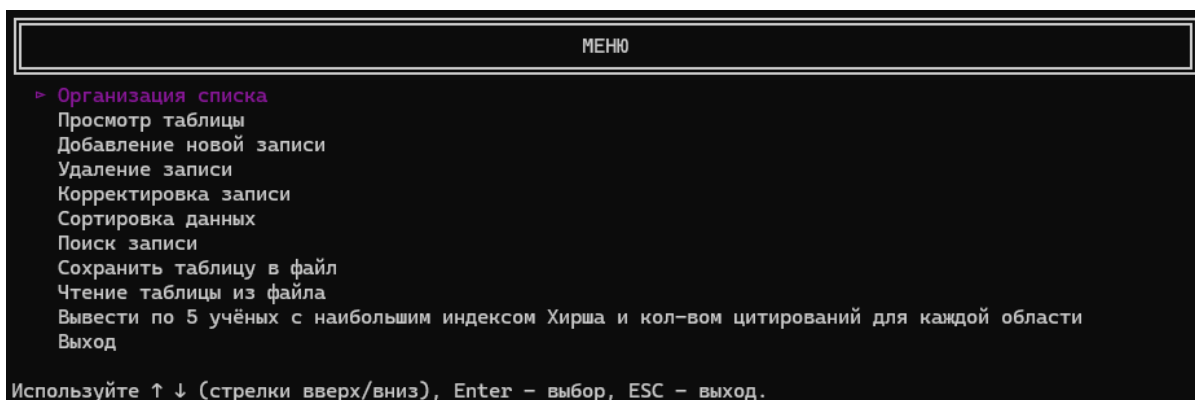


Рисунок 4.1 – Главное меню программы

Навигация в меню выполнена следующим образом:

- Клавиши ↑ (стрелка вверх) и ↓ (стрелка вниз) – перемещение по пунктам.
- Enter – выбор текущего пункта.
- Esc – выход из программы (с предварительным подтверждением, если есть несохранённые данные).

4.4 Описание работы с программой

Ниже приведены экранные формы и сообщения, которые появляются при выборе каждого пункта меню.

4.4.1 Пункт 1 – «Организация списка»

Назначение: создание нового списка учёных путём последовательного ввода данных с клавиатуры.

Последовательность действий оператора:

- Выбрать пункт 1 в главном меню.
- Если в памяти уже есть несохранённые данные, программа запросит подтверждение, путём решения простого математического примера (капча). Пример подтверждения действия путём прохождения каптчи изображён на Рис. 4.2.

- Программа последовательно запрашивает данные для каждого учёного (пример приведён на Рис. 4.3)
- После ввода всех полей программа спрашивает у пользователя, вводить ли ещё одного учёного или прекратить ввод, путём ввода 1, если необходимо ввести ещё одну запись, или 0, если необходимо завершить ввод, тем самым сохранив список в памяти компьютера.

Подтверждение действия: создание нового списка (текущие данные будут потеряны)
Для продолжения решите пример: $34 + 42 =$ |

Рисунок 4.2 – Подтверждение действия, путём решения математического примера

```
Введите ФИО: Lomonosov Mikhail Vasilyevich
Введите научную область: Natural science
Введите учёную степень: Professor
Введите количество научных статей: 260
Введите количество цитирований: 17000
Введите индекс Хирша: 50

Ввести ещё?
1. да
0. нет|
```

Рисунок 4.3 – Ввод данных об учёном с клавиатуры

После завершения ввода список сохраняется в памяти компьютера. При попытке просмотра (пункт главного меню 2) отобразится введенная информация.

4.4.2 Пункт 2 – «Просмотр таблицы»

Назначение: вывод всех записей в виде таблицы с возможностью прокрутки (скроллинга).

Действия оператора:

- Выбрать пункт 2, если список пуст, появится соответствующее сообщение (пример сообщения изображён на Рис. 4.4).

- Если список не пуст, отображается таблица и текущий диапазон записей (до 15 записей на одной странице). Пример вывода таблицы приведён на Рис. 4.5.
- Управление просмотром производится с помощью клавиш-стрелок, а при помощи кнопки Escare происходит возвращение в главное меню.

Список пуст!
Рекомендуемое действие: 1.
Нажмите любую клавишу для возвращения в меню...|

Рисунок 4.4 – Сообщение о пустом списке при просмотре таблицы

ID	Фамилия Имя Отчество	Учёная Степень	Область Науки	Кол-во статей	Кол-во цитирований	Индекс Хирша
1	John Michael Smith	PhD	Computer Science	45	1200	18
2	Jane Elizabeth Doe	Professor	Physics	78	3400	28
3	David Robert Johnson	Dr.Sc.	Mathematics	112	5600	37
4	Sarah Ann Williams	Associate Professor	Biology	34	890	12
5	Michael James Brown	Senior Researcher	Chemistry	67	2100	22
6	Maria Christina Jones	PhD	Medicine	23	450	8
7	William Thomas Garcia	Professor	Engineering	156	7800	42
8	Linda Mary Miller	Dr.Sc.	Psychology	89	3100	26
9	Richard Paul Davis	Associate Professor	Economics	42	1100	15
10	Patricia Susan Rodriguez	Senior Researcher	Sociology	38	950	13
11	Charles Joseph Martinez	PhD	Computer Science	112	4900	33
12	Barbara Elizabeth Wilson	Professor	Physics	134	6700	39
13	Thomas Christopher Anderson	Dr.Sc.	Mathematics	65	1700	20
14	Jennifer Lynn Taylor	Associate Professor	Biology	29	720	10
15	Christopher Paul Thomas	Senior Researcher	Chemistry	58	1650	19

Записи: 1-15 из 70.
Используйте:
← (предыдущая страница) → (следующая страница)
↑ (предыдущая запись) ↓ (следующая запись)
ESC (выход)

Рисунок 4.5 – Пример вывода таблицы

4.4.3 Пункт 3 – «Добавление новой записи»

Назначение: добавление одного учёного в начало или конец списка.

Действия оператора:

- Выбрать пункт 3. Появляется подменю «добавления записи» (пример подменю приведён на Рис. 4.6)
- Выбрать необходимый вариант (например, «Добавить в конец»).

- Программа запрашивает данные нового учёного аналогично п. 4.4.1. ID присваивается автоматически (максимальный существующий ID увеличивается на 1).
- После успешного добавления программа возвращается в главное меню. При успешном добавлении никаких дополнительных сообщений не выводится (информация сохраняется в списке). При попытке добавить запись в пустой список операция также выполняется корректно.

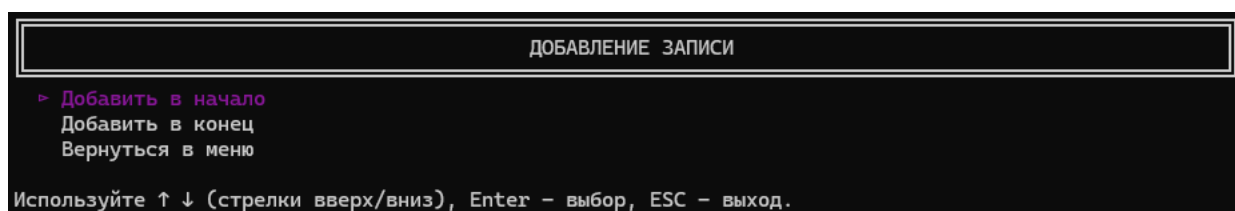


Рисунок 4.6 – Подменю «добавление записи»

4.4.4 Пункт 4 – «Удаление записи»

Назначение: удаление одной записи по ID или всех записей.

Действия оператора:

- Выбрать пункт 4
- Появляется запрос, изображённый на Рис. 4.7.
- Вводится целое число. 0 – удаление всех записей. При удалении всех записей программа потребует пройти проверку. Аналогичная проверка изображена в п. 4.4.1. При правильном ответе все записи удаляются, выводится сообщение об успешном удалении всего списка (пример изображён на Рис. 4.8), а список становится пустым. При неправильном ответе удаление отменяется. Положительное число (например 5) – обозначает удаление записи с уникальным ID, равным 5. Если запись не найдена, выводится соответствующее сообщение (пример сообщения о ненайденной записи изображён на Рис. 4.9), иначе программа удалит запись с соответствующим ID и выведет сообщение об

успешном удалении (пример сообщения об успешном удалении изображён на Рис. 4.10).

Результат: список обновляется, идентификаторы оставшихся записей не изменяются (не перенумеровываются). При удалении всех записей счётчик ID сбрасывается на значение, равное 1.

Введите ID удаляемой записи или 0 (удалить все) -> |

Рисунок 4.7 – Запрос на ввод ID для удаления записи

Подтверждение действия: удаление ВСЕХ записей
Для продолжения решите пример: $12 + 36 = 48$
Проверка пройдена. Выполняем...
70 записей удалено
Нажмите любую клавишу для возвращения в меню...|

Рисунок 4.8 – Успешное удаление всего списка

Введите ID удаляемой записи или 0 (удалить все) -> 100
Запись с ID '100' не найден
Ошибка при удалении!
Нажмите любую клавишу для возвращения в меню...|

Рисунок 4.9 – Сообщение о ненайденной для удаления записи

Введите ID удаляемой записи или 0 (удалить все) -> 5
1 записей удалено
Нажмите любую клавишу для возвращения в меню...|

Рисунок 4.10 – Успешное удаление записи

4.4.5 Пункт 5 – «Корректировка записи»

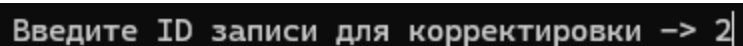
Назначение: изменение одного или нескольких полей существующей записи.

Действия оператора:

- Выбрать пункт 5.
- Программа запрашивает ID записи для редактирования (пример запроса приведён на Рис. 4.11)
- Если запись найдена, отображается подменю выбора поля для редактирования (пример подменю выбора поля для редактирования изображён на Рис. 4.12).
- Выбрать нужное поле (например, «Кол-во статей»). Программа запросит новое значение (при этом для числовых выполняется проверка на целое число)
- После успешного изменения выводится сообщение (пример сообщения об успешном изменении записи изображён на рис. 4.13)
- При выборе «В меню» или нажатии Esc изменения сохраняются и происходит выход в главное меню.

Особенности оператора:

- Можно изменять поля по одному; для изменения нескольких полей нужно повторно войти в пункт 5.
- ID изменить нельзя.
- Если введён ID несуществующей записи, программа выводит соответствующее сообщение (пример сообщения о ненайденной записи изображён на Рис. 4.14)



Введите ID записи для корректировки -> 2|

Рисунок 4.11 – Запрос ID записи для корректировки

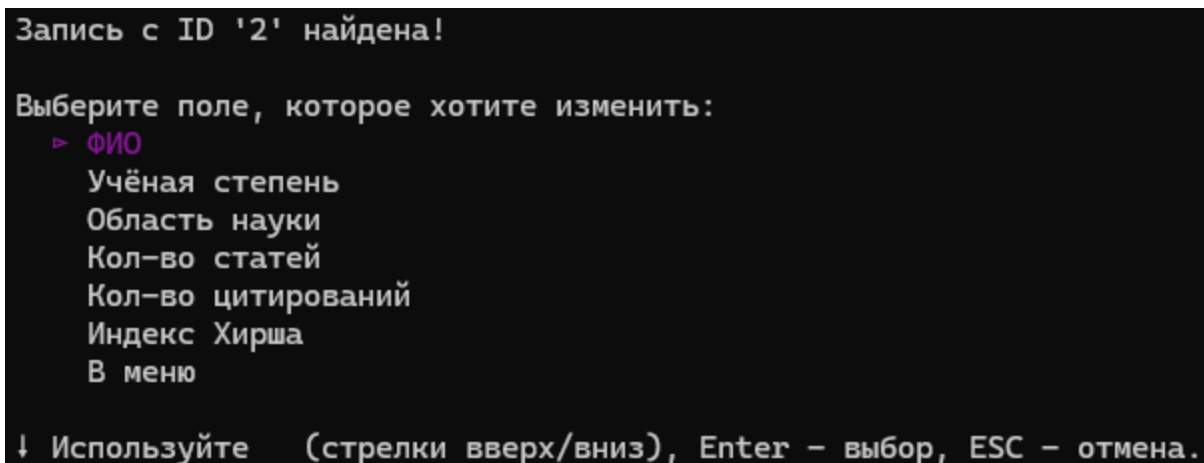


Рисунок 4.12 – Подменю выбора поля для редактирования

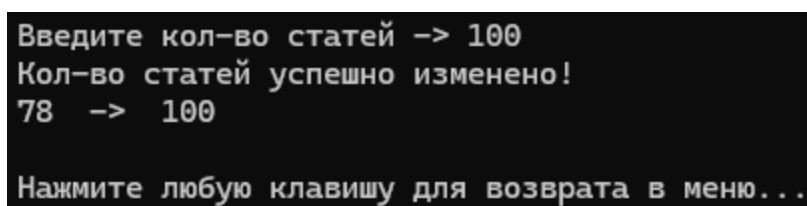


Рисунок 4.13 – Сообщение об успешном изменении записи

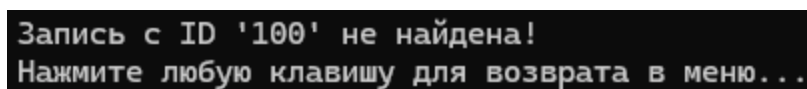


Рисунок 4.14 – Сообщение о ненайденной записи

4.4.6 Пункт 6 – «Сортировка данных»

Назначение: сортировка всех записей по выбранному полю в порядке возрастания или убывания.

Действия оператора:

- Выбрать пункт 6. Если в списке менее двух записей, выводится сообщение об отсутствии необходимости в сортировке (пример сообщения об отсутствии необходимости в сортировке изображён на Рис. 4.15), затем происходит возврат в главное меню.

- Иначе отображается подменю выбора поля, по которому будет выполняться сортировка (пример подменю для выбора поля для сортировки изображён на Рис. 4.16).
- Выбрать поле (например, «По индексу Хирша»).
- Появляется подменю выбора порядка сортировки, изображённое на Рис. 4.17.
- Выбрать порядок (например, «По убыванию»).
- Программа выполняет сортировку и выводит сообщение об успешной сортировке, изображённое на Рис. 4.18.

При повторном просмотре таблицы (пункт 2) записи отображаются в отсортированном порядке.

Список не нуждается в сортировке или пуст.
Нажмите любую клавишу для возвращения в меню...

Рисунок 4.15 – Сообщение об отсутствии необходимости в сортировке

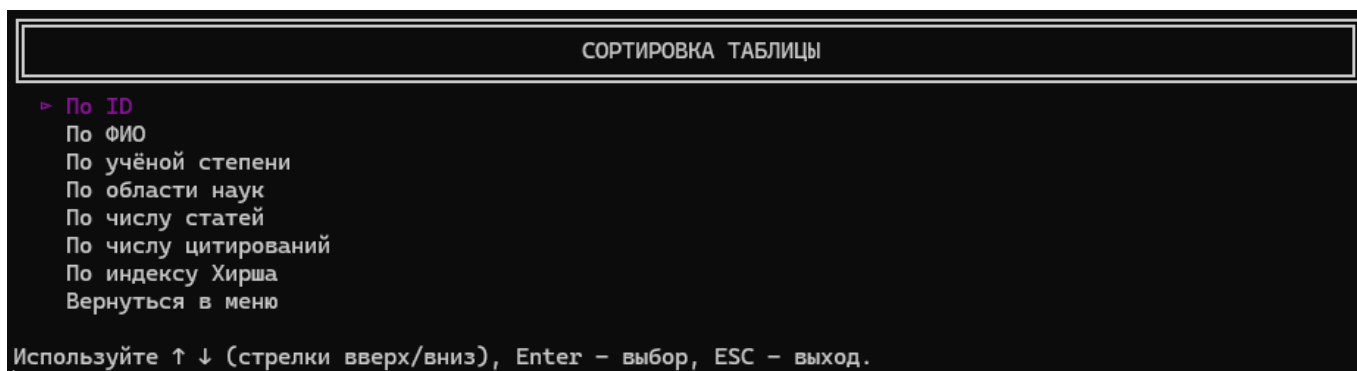


Рисунок 4.16 – Подменю для выбора поля для сортировки

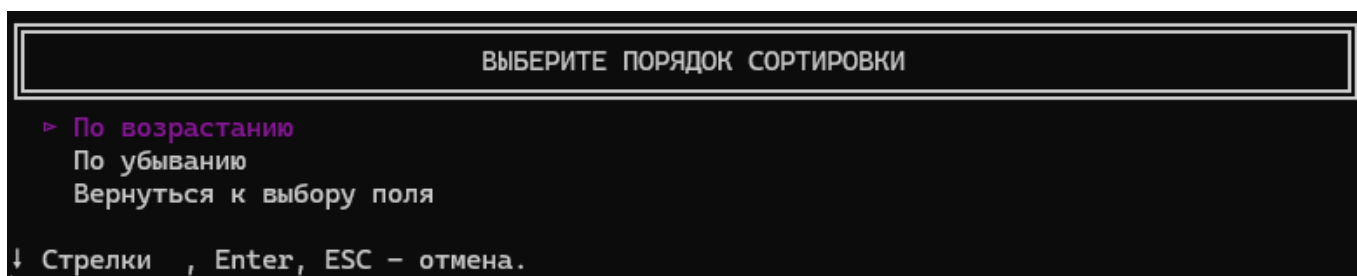
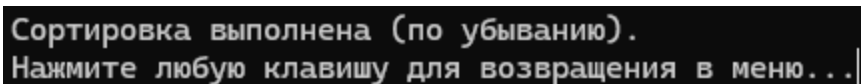


Рисунок 4.17 – Подменю для выбора порядка сортировки



Сортировка выполнена (по убыванию).
Нажмите любую клавишу для возвращения в меню...|

Рисунок 4.18 – Сообщение об успешной сортировке

4.4.7 Пункт 7 – «Поиск записи»

Назначение: поиск записей по точному совпадению значения в указанном поле.

Действия оператора:

- Выбрать пункт 7.
- Отображается подменю выбора поля (аналогично сортировке). Подменю выбора поля поиска изображено на Рис. 4.19.
-
- Выбрать поле (например, «По индексу Хирша»). Программа запрашивает значение, которое она будет искать. Пример запроса значения для поиска приведён на Рис. 4.20.
- Выполняется поиск. Результат выводится в виде таблицы. Таблица найденных значений изображена на Рис. 4.21
- Если совпадений нет, выводится сообщение, изображённое на Рис. 4.22.

Особенности:

- Поиск регистрозависимый (сравнение строк происходит с учётом регистра).
- При поиске по числовым полям (ID, статьи, цитирования, индекс Хирша) проверяется ввод целого положительного числа.

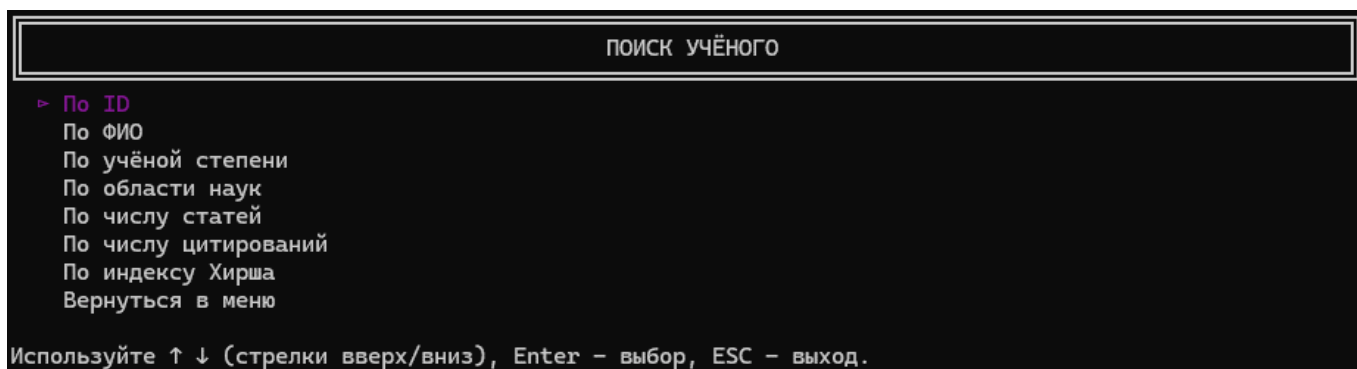


Рисунок 4.19 – Подменю выбора поля поиска

Введите Индекс Хирша -> 24

Рисунок 4.20 – Запрос значения для поиска

ID	Фамилия Имя Отчество	Учёная Степень	Область Науки	Кол-во статей	Кол-во цитирований	Индекс Хирша
18	Karen Lee Martin	Dr.Sc.	Psychology	73	2548	24
63	Teresa Lynn Gray	Dr.Sc.	Mathematics	79	2678	24

Найдено 2 записей с Индексом Хирша '24'
Нажмите любую клавишу для продолжения...

Рисунок 4.21 – Таблица найденных значений

Найдено 0 записей с Индексом Хирша '100'
Нажмите любую клавишу для продолжения...

Рисунок 4.22 – Сообщение об отсутствии совпадений

4.4.8 Пункт 8 – «Сохранить таблицу в файл»

Назначение: экспорт текущего списка в текстовый (.txt) или бинарный (.bin) файл.

Действия оператора:

- Выбрать пункт 8.
- Появляется подменю для выбора типа файла, в который будет производиться экспорт таблицы (подменю выбора типа файла изображено на Рис. 4.23).
- Выбрать нужный формат.
- Программа запрашивает имя файла. Если пользователь не укажет расширение файла, или укажет его неверно, то программа автоматически подставит

расширение в конец введённой строки. Запрос имени файла изображён на Рис. 4.24.

- Выполняется запись таблицы в файл. При успехе программа выведет сообщение об успешной записи файла (см. Рис. 4.25), и наоборот, при ошибке создания файла выведет соответствующее сообщение (см. Рис. 4.26).

Формат записи в текстовый и бинарный файл различается. В бинарном файле запись представляет собой последовательную запись структур `struct scientist`, а текстовый файл представляет каждую запись в отдельной строке с разделителем «табуляции» (`\t`).

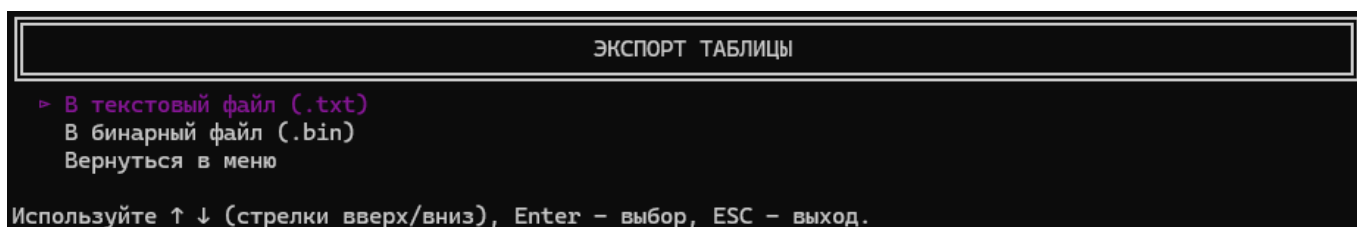


Рисунок 4.23 – Подменю выбора типа файла

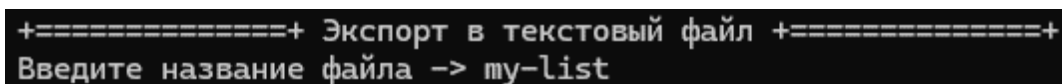


Рисунок 4.24 – Запрос имени файла

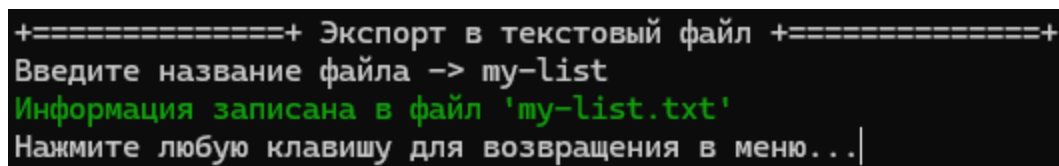


Рисунок 4.25 – Сообщение об успешной записи файла

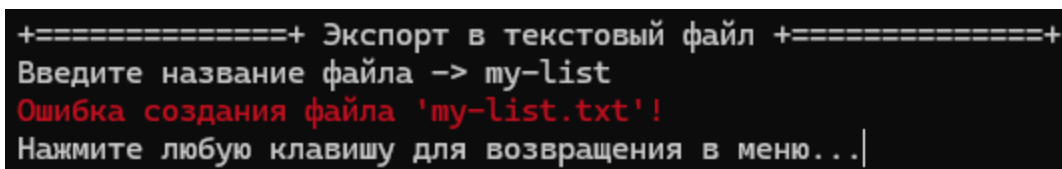


Рисунок 4.26 – Сообщение об ошибке записи файла

4.4.9 Пункт 9 – «Чтение таблицы из файла»

Назначение: импорт списка из ранее сохранённого файла (текстового или бинарного).

Действия оператора:

- Выбрать пункт 9.
- Если в памяти уже есть несохранённые данные, выводится проверка с предупреждением о потере данных (аналогично п. 4.4.1).
- Появляется подменю выбора типа файла (см. Рис. 4.27).
- Выбрать формат. Программа запрашивает имя файла (см Рис. 4.28). Аналогично пункту меню 8, программа проверяет наличие корректного расширения для введённой строки.
- Если файл существует и имеет корректную структуру, данные считываются и добавляются в список (предыдущий список уничтожается), затем выводится сообщение об успешном импорте из файле (см. Рис. 4.29). Иначе, если файл не существует или имеет в себе некорректное представление данных, то программа выводит сообщение об ошибке (см. Рис. 4.30).

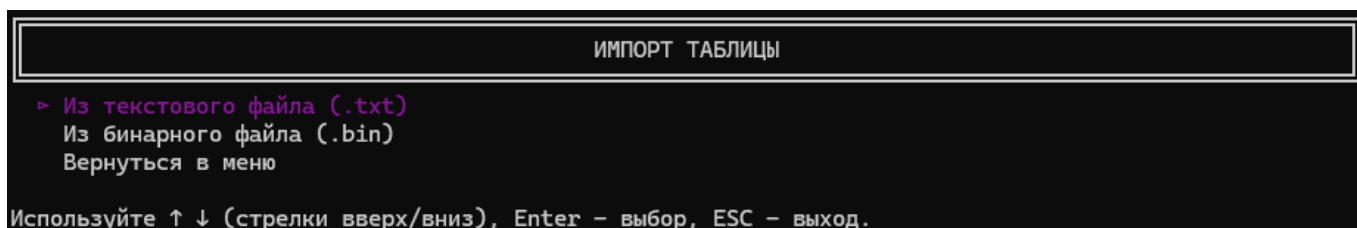


Рисунок 4.27 – Подменю выбора типа файла

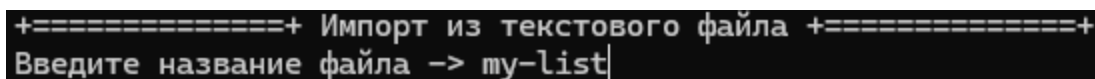


Рисунок 4.28 – Запрос имени файла

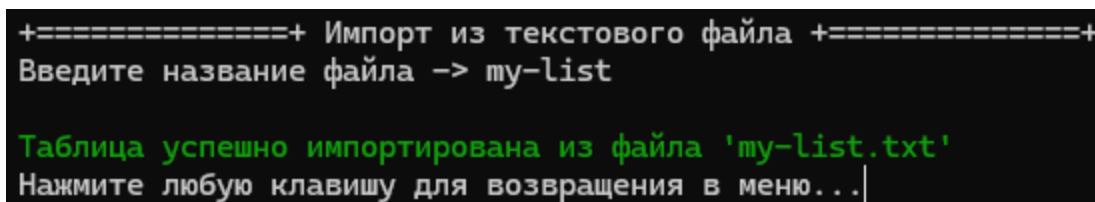


Рисунок 4.29 – Сообщение об успешном прочтении таблицы из файла


```

+=====+ Импорт из текстового файла +=====+
Введите название файла -> my-list

Возникла ошибка во время чтения файла 'my-list.txt'!
Нажмите любую клавишу для возвращения в меню...|

```

Рисунок 4.30 – Сообщение об ошибке прочтения таблицы из файла

4.4.10 Пункт 10 – «Вывести по 5 учёных с наибольшим индексом Хирша и количеством цитирований для каждой области»

Назначение: для каждой научной области вывод топ-5 учёных по индексу Хирша и топ-5 по количеству цитирований, результат сохраняется в файл 10results.txt

- Выбрать пункт 10.
- Если список пуст, выводится сообщение, что список пуст (см. Рис. 4.31).
- Программа анализирует список, группирует по области, сортирует каждую группу по индексу Хирша и по цитированиям, выводит на экран таблицы для каждой области и одновременно записывает все строки в файл 10results.txt.
- На экране последовательно отображаются блоки (пример одного из блока приведён на Рис. 4.32)
- После отображения всех блоков будет выведено сообщение об экспорте совокупности всех блоков в текстовый файл 10results.txt (см. Рис. 4.33).

```

Список пуст!
Рекомендуемое действие: 1.

Нажмите любую клавишу для возвращения в меню...|

```

Рисунок 4.31 – Сообщение о пустом списке

===== Computer Science =====

Топ-5 по Хиршу:

ID	Фамилия Имя Отчество	Учёная Степень	Область Науки	Кол-во статей	Кол-во цитирований	Индекс Хирша
1	John Michael Smith	PhD	Computer Science	45	1200	18
61	Lorraine Marie Torres	PhD	Computer Science	96	3850	29
21	Donald Edward Harris	PhD	Computer Science	98	3700	29
31	Kenneth Ray Adams	PhD	Computer Science	104	4250	31
51	Kathleen Marie Morgan	PhD	Computer Science	108	4600	32

Топ-5 по цитированиям:

ID	Фамилия Имя Отчество	Учёная Степень	Область Науки	Кол-во статей	Кол-во цитирований	Индекс Хирша
1	John Michael Smith	PhD	Computer Science	45	1200	18
21	Donald Edward Harris	PhD	Computer Science	98	3700	29
61	Lorraine Marie Torres	PhD	Computer Science	96	3850	29
31	Kenneth Ray Adams	PhD	Computer Science	104	4250	31
51	Kathleen Marie Morgan	PhD	Computer Science	108	4600	32

Рисунок 4.32 – Один из полученных блоков (по обл. “Computer Science”)

```
Результаты в файле '10results.txt'
Нажмите любую клавишу для возвращения в меню...
```

Рисунок 4.33 – Сообщение об экспорте совокупности всех блоков в текстовый файл
10results.txt

4.4.11 Пункт 11 – «Выход из программы»

Назначение: завершение работы программы.

Действия оператора:

- Выбрать пункт 11 или нажать Esc в главном меню.
- Если в памяти есть несохранённые данные (после добавления, редактирования, удаления, сортировки, импорта, но без последующего экспорта), выводится проверка, аналогичная п. 4.4.1 (см. Рис 4.34).
- Если все данные сохранены, программа сразу завершается.

```
Подтверждение действия: выход без сохранения
Для продолжения решите пример: 5 + 19 = 24
Проверка пройдена. Выполняем...

Удалено 70 записей. Выход...
```

Рисунок 4.34 – Сообщение об успешном завершении выполнения программы

5 ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы была достигнута поставленная цель: разработана программа на языке C для ведения базы данных об учёных с использованием линейного двунаправленного списка. Программа обеспечивает полный набор операций: создание, просмотр, добавление, удаление, редактирование, сортировку, поиск, импорт/экспорт записей, а также специальную функцию вывода топ-5 учёных по наукометрическим показателям для каждой области науки.

Решены следующие задачи:

- Разработаны структуры данных `struct scientist` и `struct list`, а также реализованы функции `addFirst`, `addLast`, `freeMemory`, обеспечивающие динамическое управление памятью.
- Реализован алгоритм пузырьковой сортировки (функция `sortTable`) с возможностью выбора поля для упорядочивания; это позволило гибко сортировать список по любому из семи атрибутов.
- Создан линейный поиск (функция `findScientist`), который корректно обрабатывает как числовые, так и строковые критерии, выводя все подходящие записи.
- Реализован постраничный вывод таблицы с клавиатурной навигацией (`viewList`), что значительно повышает удобство работы при большом количестве записей.
- Обеспечен импорт/экспорт данных в текстовые и бинарные файлы; функция `checkFileExt` автоматически добавляет расширение, предотвращая ошибки пользователя.
- Разработана сложная функция `top5ByArea`, которая динамически формирует временные списки для каждой области, выполняет двойную сортировку и сохраняет результат в файл `0results-success.txt`, а также удаляет дубликаты строк.

Программа полностью реализует все требования варианта №4, утверждённого 18.02.2026. Интерфейс является консольным, интуитивно понятным, все операции сопровождаются подсказками и сообщениями об ошибках.

Дальнейшее использование данной программы может представлять собой основу для учебных проектов по структурам данных, а также может быть расширена до полноценной информационной системы с графическим интерфейсом, поддержкой сетевого доступа или интеграцией с SQL-базами данных. Полученные алгоритмы (сортировка обменом содержимого узлов, линейный поиск в списке, временные списки для группировки) могут быть применены в других задачах, требующих обработки динамических баз.

6 СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- Керниган Б. У., Ритчи Д. М. Язык программирования С. – 2-е изд. – М. : Вильямс, 2009. – 304 с.
- Дейтел Х. М., Дейтел П. Дж. Как программировать на С. – 5-е изд. – М. : Бином-Пресс, 2008. – 1024 с.
- Вирт Н. Алгоритмы и структуры данных. – СПб. : Невский диалект, 2001. – 352 с.
- ГОСТ 19.701-90 (ИСО 5807-85). Единая система программной документации. Схемы алгоритмов, программ, данных и систем. – М. : Издательство стандартов, 1990.
- Страуструп Б. Язык программирования С++. – 4-е изд. – М. : Диалектика, 2019. – 1368 с. (использован для ознакомления с концепцией списков в С++).
- Техническое задание на курсовой проект по дисциплине «Программирование» кафедры ИТС СевГУ, вариант №4, утверждённое 18.02.2026.
- Документация по Visual Studio Code : [Электронный ресурс]. – Режим доступа: <https://code.visualstudio.com/docs> (дата обращения: 04.06.2026).

ПРИЛОЖЕНИЕ А

Текст программы

/*

+++++

Севастопольский государственный университет
Кафедра "Информационные технологии и системы"

Программа для работы с базой сведений об учёных
Текст программы

РАЗРАБОТАЛ

Студент гр. ИИ/б-25-6-о

Заварзин А.В.

2026

+++++

Программа работает с базой данных об учёных, которая считывается из
текстового файла. Каждая строка файла содержит запись об одном учёном,
для которой указывается ФИО учёного, научная область,
учёная степень, количество статей и цитирований, индекс Хирша.

Основные функции программы:

- вывод базы на экран
- добавление записи об учёном в базу;
- исправление записи об учёном в базе;
- удаление записи об учёном из базы;
- поиск записи об учёном в базе;
- сортировка записей об учёных;

Вариант задания 4. Утверждено 18.02.2026

Среда программирования Visual Studio Code version 1.123.0

Дата последней коррекции: 4.06.2026.

Версия 0.6.64

```
+++++
+++++
```

```
*/
```

```
#include <stdio.h>
#include <conio.h>
#include <math.h>
#include <stdlib.h>
#include <time.h>
#include <string.h>
#include <windows.h>
#define ESC (int)27
#define UP (int)72
#define DOWN (int)80
#define LEFT (int)75
#define RIGHT (int)77
#define ENTER (int)13
#define PAGESIZE (int)15
int nextID = 1;
```

```
//-----структуры данных-----
-----
```

```
struct scientist {          // Структура для хранения данных об
учёном
    int id;                 // ID
    char name[76];          // ФИО
    char area[26];          // Научная область
    char degree[26];        // Учёная степень
    int articles;           // Количество статей
    int quotes;             // Количество цитирований
    int hirshIndex;         // Индекс Хирша
};
```

```
struct list {              // Элемент 2- направленного списка
```

```

    struct scientist info;    // Данные об учёном
    struct list *prev;        // Указатель на элемент слева
(предыдущий)
    struct list *next;        // Указатель на элемент справа
(следующий)
};

//-----прототипы функций-----
-----

struct scientist readData();
struct list *createListFromKeyboard(struct list *);
struct list *addFirst(struct list *, const struct scientist);
struct list *addLast(struct list *, const struct scientist);
void viewList(struct list*);
int deleteNode(struct list **, int);
struct list *editElement(struct list *, int);
struct list *sortTable(struct list *, const int, const int);
void findScientist(struct list *, const int);
void exportToTxt(struct list *);
void exportToBin(struct list *);
struct list *importFromTxt(struct list *);
struct list *importFromBin(struct list *);
void top5ByArea(struct list *);

void printTableHeader();
void printNode(struct list *);
int charToInt(char);
int isNumber(char *);
void checkFileExt(char *, const char *);
int compareScientist(const struct scientist *, const struct scientist
*, const int);
int captcha(const char *);

int showMenu(const char *, const char *[], int );

```



```

int inputInt(const char *);
void handleSort(struct list *);
void handleFind(struct list *);
void handleExport(struct list *, int *);
void handleImport(struct list **, int *);
void handleAdd(struct list **, int *);

//-----главная функция-----
-----

int main() {
    srand(time(NULL));
    struct list *head = NULL;
    int saved = 0;

    const char *mainMenuItems[] = {
        "Организация списка",
        "Просмотр таблицы",
        "Добавление новой записи",
        "Удаление записи",
        "Корректировка записи",
        "Сортировка данных",
        "Поиск записи",
        "Сохранить таблицу в файл",
        "Чтение таблицы из файла",
        "Вывести по 5 учёных с наибольшим индексом Хирша и кол-вом
цитирований для каждой области",
        "Выход"
    };

    int mainMenuSize = sizeof(mainMenuItems) /
sizeof(mainMenuItems[0]);

    while (1) {
        int choice = showMenu("МЕНЮ", mainMenuItems, mainMenuSize);
        if (choice == -1) choice = 10;
    }
}

```

```

switch (choice) {
    case 0:
        if (head && !saved && !captcha("создание нового списка
(текущие данные будут потеряны)))
            break;
        head = createListFromKeyboard(head);
        saved = 0;
        break;

    case 1:
        if (head) viewList(head);
        else {
            system("cls");
            printf("Список пуст!\nРекомендуемое действие:
1.\n\nНажмите любую клавишу...");
            getch();
        }
        break;

    case 2:
        handleAdd(&head, &saved);
        break;

    case 3:
        if (!head) {
            printf("Список пуст!\nРекомендуемое действие:
1.");

            getch();
            break;
        }
        system("cls");
        int idToDel = inputInt("Введите ID удаляемой записи
или 0 (удалить все) -> ");

```

```

        if (idToDel == 0 && !saved && !captcha("удаление ВСЕХ
записей")) {
            printf("Нажмите любую клавишу...");
            getch();
            break;
        }
        int delCount = deleteNode(&head, idToDel);
        if (delCount == -1) printf("Ошибка при удалении!\n");
        else printf("%d записей удалено\n", delCount);
        printf("Нажмите любую клавишу...");
        getch();
        break;

    case 4:
        if (!head) {
            printf("Список пуст!");
            getch();
            break;
        }
        system("cls");
        int idToEdit = inputInt("Введите ID записи для
корректировки -> ");
        head = editElement(head, idToEdit);
        saved = 0;
        break;

    case 5:
        handleSort(head);
        break;

    case 6:
        handleFind(head);
        break;

    case 7:

```

```

        handleExport(head, &saved);
        break;

    case 8:
        handleImport(&head, &saved);
        break;

    case 9:
        if (!head) {
            printf("Список пуст!\n");
            getch();
            break;
        }
        top5ByArea(head);
        getch();
        break;

    case 10:
        if (head && !saved && !captcha("выход без
сохранения")) break;
        int cnt = deleteNode(&head, 0);
        printf("\nУдалено %d записей. Выход...\n", cnt);
        Sleep(3000);
        return 0;

    default:
        printf("\nОшибка: неизвестный пункт!");
        break;
}

}

}

/// @brief Функция вывода шапки таблицы
void printTableHeader() {

```

```

printf("=====
=====
===== \n")
;

printf("|| ID ||                      Фамилия Имя
Отчество ||                      Учёная
Степень ||          Область Науки || Кол-во статей || Кол-во
цитирований || Индекс Хирша || \n");

printf("=====
=====
===== \n")
;
}

/// @brief Функция печати очередного учёного
/// @param node указатель на элемент, который будет напечатан
void printNode(struct list *node) {
    printf("|| %-4d || %-75s || %-25s || %-25s || %-13d || %-18d || %-12d
|| \n",

        node->info.id,
        node->info.name,
        node->info.degree,
        node->info.area,
        node->info.articles,
        node->info.quotes,
        node->info.hirshIndex);

    printf("=====
=====
===== \n")
;
}

/// @brief функция установки (обновления) уникального ID после
изменений базы
/// @param head указатель на первый элемент списка

```

```

/// @param _mode режим
void setNewID(struct list *head, const int _mode) {
    int maxID = -1;
    struct list *temp = head;
    if (_mode == 0) {
        nextID = 1;
    } else if (_mode == 1) {
        if (head == NULL) {
            nextID = 1;
            return;
        }
        for (; temp != NULL; temp = temp->next) {
            if (maxID < temp->info.id) nextID = temp->info.id + 1;
        }
    }
}

/// @brief функция перевода char -> int
/// @param ch символ
/// @return int представление символа
int charToInt(char ch) {
    switch (ch) {
        case '0': return 0;
        case '1': return 1;
        case '2': return 2;
        case '3': return 3;
        case '4': return 4;
        case '5': return 5;
        case '6': return 6;
        case '7': return 7;
        case '8': return 8;
        case '9': return 9;
        default: return -1;
    }
}

```

```

/// @brief функция проверки строки на число
/// @param _str строка для проверки
/// @return `-1` - если строка не является целым числом из отрезка
[0:] иначе число
int isNumber(char *_str) {
    int itog = 0;
    for(int i = 0; i < strlen(_str); i++) {
        if ('0' <= _str[i] && _str[i] <= '9') itog +=
charToInt(_str[i]) * pow(10, strlen(_str) - 1 - i);
        else return -1;
    }
    return itog;
}

/// @brief Функция чтения данных с клавиатуры
/// @return Данные об учёном
struct scientist readData() {
    struct scientist data;
    char buffer[25];
    int flag = 1;

    data.id = nextID++;

    printf("Введите ФИО: ");
    fgets(data.name, 75, stdin);
    data.name[strcspn(data.name, "\n")] = '\0';

    printf("Введите научную область: ");
    fgets(data.area, 25, stdin);
    data.area[strcspn(data.area, "\n")] = '\0';

    printf("Введите учёную степень: ");
    fgets(data.degree, 25, stdin);
    data.degree[strcspn(data.degree, "\n")] = '\0';

```

```

while (flag) {
    printf("Введите количество научных статей: ");
    fgets(buffer, 25, stdin);
    buffer[strcspn(buffer, "\n")] = '\0';

    if (isNumber(buffer) != -1) {
        data.articles = isNumber(buffer);
        flag = 0;
    } else {
        printf("\nВведено некорректное число! Повторите ввод\n");
    }
}
flag = 1;

while (flag) {
    printf("Введите количество цитирований: ");
    fgets(buffer, 25, stdin);
    buffer[strcspn(buffer, "\n")] = '\0';

    if (isNumber(buffer) != -1) {
        data.quotes = isNumber(buffer);
        flag = 0;
    } else {
        printf("\nВведено некорректное число! Повторите ввод\n");
    }
}
flag = 1;

while (flag) {
    printf("Введите индекс Хирша: ");
    fgets(buffer, 25, stdin);
    buffer[strcspn(buffer, "\n")] = '\0';

    if (isNumber(buffer) != -1) {

```



```

        data.hirshIndex = isNumber(buffer);
        flag = 0;
    } else {
        printf("\nВведено некорректное число! Повторите ввод\n");
    }
}

return data;
}

/// @brief Функция добавления в начало списка
/// @param head начало списка
/// @param data данные об учённом
/// @return Указатель на начало списка (указатель на добавленный в
начало элемент)
struct list *addFirst(struct list *head, const struct scientist data)
{
    struct list *temp;
    temp = (struct list*)malloc(sizeof(struct list));
    temp->info = data;
    temp->next = head;
    temp->prev = NULL;
    return temp;
}

/// @brief Функция добавления в конец списка
/// @param head начало списка
/// @param data данные об учённом
/// @return Указатель на начало списка
struct list *addLast(struct list *head, const struct scientist data) {
    struct list *temp, *e;
    temp = (struct list*)malloc(sizeof(struct list));
    temp->info = data;
    temp->next = NULL;

```

```

    if (head == NULL) {
        head = temp;
        head->prev = NULL;
    } else {
        for (e = head; e->next != NULL; e = e->next);
        temp->prev = e;
        e->next = temp;
    }
    return head;
}

/// @brief Функция создания списка с клавиатуры
/// @param head указатель на начало списка
/// @return Указатель на начало списка
struct list *createListFromKeyboard(struct list *head) {
    int choice = 1;

    setNewID(head, 1);

    while (choice) {
        system("cls");
        if (head == NULL) head = addFirst(head, readData());
        else addLast(head, readData());

        printf("\nВвести ещё? \n1. да \n0. нет");
        choice = getch();
        choice -= 48;
    }

    return head;
}

/// @brief Функция вывода таблицы со скроллингом
/// @param head - Указатель на начало списка
void viewList(struct list *head) {

```

```

int total = 0, start = 0;
struct list *cur = head;
char key;

for (; cur != NULL; cur = cur->next) total++;

while (1) {
    system("cls");
    struct list *temp = head;
    int idx = 0, printed = 0;

    printTableHeader();

    for (; temp != NULL, idx < start; temp = temp->next) idx++;

    for (; temp != NULL && printed < PAGE_SIZE; temp = temp->next,
printed++) printNode(temp);

    int first = start + 1;
    int last = (start + printed < total) ? start + printed :
total;

    printf("\nЗаписи: %d-%d из %d. \nИспользуйте:\n← (предыдущая
страница)\t → (следующая страница) \n↑ (предыдущая запись)\t ↓
(следующая запись) \nESC (выход)", first, last, total);

    printf("\n\n[ * ] Одна страница занимает %d записей",
PAGE_SIZE);

    key = getch();

    if (key == LEFT) {
        int new_start = start - PAGE_SIZE;
        if (new_start < 0) new_start = 0;
        start = new_start;
    } else if (key == RIGHT) {
        int new_start = start + PAGE_SIZE;

```

```

        int max_start = total - PAGESIZE;
        if (max_start < 0) max_start = 0;
        if (new_start > max_start) new_start = max_start;
        start = new_start;
    } else if (key == UP) {
        int new_start = start - 1;
        if (new_start < 0) new_start = 0;
        start = new_start;
    } else if (key == DOWN) {
        int new_start = start + 1;
        int max_start = total - PAGESIZE;
        if (max_start < 0) max_start = 0;
        if (new_start > max_start) new_start = max_start;
        start = new_start;
    } else if (key == ESC) {
        return;
    }
};
}

```

/// @brief Проверяет и добавляет расширение файла, если его нет

```

void checkFileExt(char *filename, const char *ext) {
    int len = strlen(filename);
    if (len < 4 || strcmp(filename + len - 4, ext) != 0) {
        strcat(filename, ext);
    }
}

```

/// @brief функция записи очередного учёного в файл

/// @param file указатель на файл

/// @param node указатель на записываемый элемент

```

void writeToTextFile(FILE *file, struct list *node) {
    fprintf(file, "%d\t%s\t%s\t%d\t%d\t%d\n",
        node->info.id,
        node->info.name,

```

```

        node->info.degree,
        node->info.area,
        node->info.articles,
        node->info.quotes,
        node->info.hirshIndex
    );
}

/// @brief Функция экспорта таблицы в текстовый (`.txt`) файл
/// @param head Указатель на начало списка
void exportToTxt(struct list *head) {
    char fileName[101];
    struct list *temp = head;

    printf("+=====+ Экспорт в текстовый файл
+=====+\n");

    printf("Введите название файла -> ");
    fgets(fileName, sizeof(fileName), stdin);
    fileName[strcspn(fileName, "\n")] = '\0';

    checkFileExt(fileName, ".txt");

    FILE *file = fopen(fileName, "wt");
    if (file == NULL) {
        printf("\033[31mОшибка создания файла '%s'!\n", fileName);
        printf("\033[0mНажмите любую клавишу для возвращения в
меню...");
        getch();
        return;
    }

    for (; temp != NULL; temp = temp->next) {
        writeToTextFile(file, temp);
    }
    fclose(file);
}

```

```

printf("\033[32mИнформация записана в файл '%s'\n", fileName);
printf("\033[0mНажмите любую клавишу для возвращения в меню...");
getch();
}

/// @brief Функция экспорта таблицы в `.bin` файл
/// @param head Указатель на начало списка
void exportToBin(struct list *head) {
    char fileName[101];
    printf("+=====+ Экспорт в бинарный файл
+=====+\n");
    printf("Введите название файла -> ");
    fgets(fileName, sizeof(fileName), stdin);
    fileName[strcspn(fileName, "\n")] = '\0';

    checkFileExt(fileName, ".bin");

    FILE *file = fopen(fileName, "wb");
    if (file == NULL) {
        printf("\033[31mОшибка создания файла '%s'!\n", fileName);
        printf("\033[0mНажмите любую клавишу для возвращения в
меню...");
        getch();
        return;
    }

    for (struct list *temp = head; temp != NULL; temp = temp->next) {
        fwrite(&temp->info, sizeof(struct scientist), 1, file);
    }
    fclose(file);

    printf("\033[32mТаблица успешно экспортирована в файл '%s'\n",
fileName);
    printf("\033[0mНажмите любую клавишу для возвращения в меню...");

```

```

    getch();
}

/// @brief Функция импорта таблицы из текстового (`.txt`) файла
/// @param head Указатель на начало списка
/// @return Указатель на начало списка
struct list *importFromTxt(struct list *head) {
    char fileName[101];
    struct scientist data;
    system("cls");
    printf("+=====+ Импорт из текстового файла
+=====+\n");
    printf("Введите название файла -> ");
    fgets(fileName, sizeof(fileName), stdin);
    fileName[strcspn(fileName, "\n")] = '\0';

    checkFileExt(fileName, ".txt");

    FILE *file = fopen(fileName, "rt");
    if (file == NULL) {
        printf("\n\n\033[31mФайл '%s' не найден!\n", fileName);
        printf("\033[0mНажмите любую клавишу для возвращения в
меню...");
        getch();
        return head;
    }

    while (fscanf(file, "%d\t%[^\\t]\\t%[^\\t]\\t%d\t%d\t%d\n",
        &data.id, data.name, data.degree, data.area,
        &data.articles, &data.quotes, &data.hirshIndex) ==
7) {
        head = (head == NULL) ? addFirst(head, data) : addLast(head,
data);
    }
}

```

```

fclose(file);

if (head == NULL) {
    printf("\n\n\033[31mВозникла ошибка во время чтения файла
'%s'!\n", fileName);
    printf("\033[0mНажмите любую клавишу для возвращения в
меню...\n");
    getch();
    return head;
}

setNewID(head, 1);

printf("\n\n\033[32mТаблица успешно импортирована из файла '%s'\n",
fileName);
printf("\033[0mНажмите любую клавишу для возвращения в меню...\n");
getch();

return head;
}

/// @brief Функция импорта таблицы из бинарного (`.bin`) файла
/// @param head Указатель на начало списка
/// @return Указатель на начало списка
struct list *importFromBin(struct list *head) {
    char fileName[101];
    struct scientist data;
    system("cls");
    printf("+=====+ Импорт из бинарного файла
+=====+\n");
    printf("Введите название файла -> ");
    fgets(fileName, sizeof(fileName), stdin);
    fileName[strcspn(fileName, "\n")] = '\0';

    checkFileExt(fileName, ".bin");

```



```

FILE *file = fopen(fileName, "rb");
if (file == NULL) {
    printf("\n\n\033[31mФайл '%s' не найден!\n", fileName);
    printf("\033[0mНажмите любую клавишу для возвращения в
меню...");
    getch();
    return head;
}

while (fread(&data, sizeof(struct scientist), 1, file) == 1) {
    if (head == NULL) head = addFirst(head, data);
    else head = addLast(head, data);
}

fclose(file);

if (head == NULL) {
    printf("\n\n\033[31mВозникла ошибка во время чтения файла
'%s'!\n", fileName);
    printf("\033[0mНажмите любую клавишу для возвращения в
меню...");
    getch();
    return head;
}

setNewID(head, 1);

printf("\n\033[32mТаблица успешно импортирована из файла '%s'\n",
fileName);
printf("\033[0mНажмите любую клавишу для возвращения в меню...");
getch();

return head;
}

```

```

/// @brief Функция очистки памяти
/// @param head Указатель на начало списка
/// @return Кол-во удалённых записей
int deleteNode(struct list **head, int choice) {
    if (head == NULL) return -1;

    struct list *temp = NULL;

    switch (choice) {
        case 0: {
            if (*head == NULL) {
                printf("\nСписок пуст...\n");
                return 0;
            }
            int count = 0;
            while (*head != NULL) {
                temp = *head;
                *head = (*head)->next;
                free(temp);
                count++;
            }
            *head = NULL;
            setNewID(*head, 0);
            return count;
        }

        case 1: {
            if (*head == NULL) {
                printf("\nСписок пуст, нечего удалять\n");
                return -1;
            }
            temp = *head;
            *head = (*head)->next;
            if (*head != NULL) (*head)->prev = NULL;

```

```

        free(temp);
        setNewID(*head, 1);
        return 1;
    }

    default: {
        struct list *toDel = *head;

        while (toDel != NULL && toDel->info.id != choice) toDel =
toDel->next;

        if (toDel == NULL) {
            printf("\nЗапись с ID '%d' не найдена\n", choice);
            return -1;
        }

        if (toDel->prev != NULL) toDel->prev->next = toDel->next;
        else *head = toDel->next;

        if (toDel->next != NULL) toDel->next->prev = toDel->prev;

        free(toDel);
        return 1;
    }
}

/// @brief Функция корректировки данных об учёном по его ID
/// @param head Указатель на начало списка
/// @param id ID записи, которая будет откорректирована
/// @return Указатель на начало списка
struct list *editElement(struct list *head, int id) {
    int choice = 0;           // 1..6 или 0 (меню)
    int oldInt, flag;
    char oldLine[76], buffer[25];

```

```

struct list *temp = head;

for (; temp != NULL && temp->info.id != id; temp = temp->next);

if (temp == NULL) {
    system("cls");
    printf("Запись с ID '%d' не найдена!", id);
    printf("\nНажмите любую клавишу для возврата в меню...");
    getch();
    return head;
}

const char *editItems[] = {
    "ФИО",
    "Учёная степень",
    "Область науки",
    "Кол-во статей",
    "Кол-во цитирований",
    "Индекс Хирша",
    "В меню"
};

int editSize = sizeof(editItems) / sizeof(editItems[0]);
int currentEdit = 0;

while (1) {
    system("cls");
    printf("Запись с ID '%d' найдена!\n\n", id);
    printf("Выберите поле, которое хотите изменить:\n");
    for (int i = 0; i < editSize; i++) {
        printf("  %s %s\n", (i == currentEdit) ? "\033[35m▶" : "
", editItems[i]);
        printf("\033[0m");
    }
    printf("\nИспользуйте ↑ ↓ (стрелки вверх/вниз), Enter - выбор,
ESC - выход.\n");
}

```

```

int key = getch();
if (key == 224) {
    key = getch();
    if (key == UP)
        currentEdit = (currentEdit - 1 + editSize) % editSize;
    else if (key == DOWN)
        currentEdit = (currentEdit + 1) % editSize;
} else if (key == ENTER) {
    if (currentEdit == editSize - 1) {
        choice = 0;
    } else {
        choice = currentEdit + 1;
    }
    break;
} else if (key == ESC) {
    choice = 0;
    break;
}
}

system("cls");
switch (choice) {
    case 1:
        printf("Введите ФИО -> ");
        strcpy(oldLine, temp->info.name);
        fgets(temp->info.name, 75, stdin);
        temp->info.name[strcspn(temp->info.name, "\n")] = '\0';
        printf("ФИО успешно изменено!\n");
        printf("%s -> %s\n", oldLine, temp->info.name);
        break;

    case 2:
        printf("Введите учёную степень -> ");
        strcpy(oldLine, temp->info.degree);

```

```

fgets(temp->info.degree, 25, stdin);
temp->info.degree[strcspn(temp->info.degree, "\n")] =
'\0';

printf("Учёная степень успешно изменена!\n");
printf("%s -> %s\n", oldLine, temp->info.degree);
break;

case 3:
printf("Введите область наук -> ");
strcpy(oldLine, temp->info.area);
fgets(temp->info.area, 25, stdin);
temp->info.area[strcspn(temp->info.area, "\n")] = '\0';
printf("Область наук успешно изменена!\n");
printf("%s -> %s\n", oldLine, temp->info.area);
break;

case 4:
oldInt = temp->info.articles;
flag = 1;
while (flag) {
printf("Введите кол-во статей -> ");
fgets(buffer, 25, stdin);
buffer[strcspn(buffer, "\n")] = '\0';
int val = isNumber(buffer);
if (val >= 0) {
temp->info.articles = val;
flag = 0;
} else {
printf("\nОшибка: введите целое неотрицательное
число!\n");
}
}

printf("Кол-во статей успешно изменено!\n");
printf("%d -> %d\n", oldInt, temp->info.articles);
break;

```

case 5:

```

oldInt = temp->info.quotes;
flag = 1;
while (flag) {
    printf("Введите кол-во цитирований -> ");
    fgets(buffer, 25, stdin);
    buffer[strcspn(buffer, "\n")] = '\0';
    int val = isNumber(buffer);
    if (val >= 0) {
        temp->info.quotes = val;
        flag = 0;
    } else {
        printf("\nОшибка: введите целое неотрицательное
число!\n");
    }
}
printf("Кол-во цитирований успешно изменено!\n");
printf("%d -> %d\n", oldInt, temp->info.quotes);
break;

```

case 6:

```

oldInt = temp->info.hirshIndex;
flag = 1;
while (flag) {
    printf("Введите индекс Хирша -> ");
    fgets(buffer, 25, stdin);
    buffer[strcspn(buffer, "\n")] = '\0';
    int val = isNumber(buffer);
    if (val >= 0) {
        temp->info.hirshIndex = val;
        flag = 0;
    } else {
        printf("\nОшибка: введите целое неотрицательное
число!\n");
    }
}

```

```

        }
    }
    printf("Индекс Хирша успешно изменён!\n");
    printf("%d -> %d\n", oldInt, temp->info.hirshIndex);
    break;

default: // choice == 0 или некорректный
    // Ничего не делаем, просто возвращаемся
    break;
}

printf("\nНажмите любую клавишу для возврата в меню...");
getch();
return head;
}

/// @brief Функция поиска данных в таблице
/// @param head указатель на начало списка
/// @param choice поле для поиска
void findScientist(struct list *head, const int choice) {
    struct list *temp = head;
    int key, count = 0, flag = 1;
    char buffer[75];

    system("cls");

    switch (choice) {
        case 1:
            while (flag) {
                printf("Введите ID -> ");
                fgets(buffer, 75, stdin);
                buffer[strcspn(buffer, "\n")] = '\0';
                if (isNumber(buffer) != -1) key = isNumber(buffer);
                else {

```



```

        printf("\nВведено некорректное число в поле 'ID'!
Повторите ввод снова\n");
        continue;
    }
    flag = 0;
}
printTableHeader();
for (; temp != NULL; temp = temp->next) {
    if (temp->info.id == key) {
        printNode(temp);
        count++;
    }
}
printf("Найдено %d записей с ID '%d'", count, key);
return;

case 2:
    printf("Введите ФИО -> ");
    fgets(buffer, sizeof(buffer), stdin);
    buffer[strcspn(buffer, "\n")] = 0;
    printTableHeader();
    for (; temp != NULL; temp = temp->next) {
        if (strcmp(temp->info.name, buffer) == 0) {
            printNode(temp);
            count++;
        }
    }
    printf("Найдено %d записей с ФИО '%s'", count, buffer);
    return;

case 3:
    printf("Введите Учёную степень -> ");
    fgets(buffer, sizeof(buffer), stdin);
    buffer[strcspn(buffer, "\n")] = 0;
    printTableHeader();

```

```

    for (; temp != NULL; temp = temp->next) {
        if (strcmp(temp->info.degree, buffer) == 0) {
            printNode(temp);
            count++;
        }
    }
    printf("Найдено %d записей с Учёной степенью '%s'", count,
buffer);

    return;

case 4:
    printf("Введите Область наук -> "); fgets(buffer,
sizeof(buffer), stdin);
    buffer[strcspn(buffer, "\n")] = 0;
    printTableHeader();
    for (; temp != NULL; temp = temp->next) {
        if (strcmp(temp->info.area, buffer) == 0) {
            printNode(temp);
            count++;
        }
    }
    printf("Найдено %d записей с Областью наук '%s'", count,
buffer);

    return;

case 5:
    while (flag) {
        printf("Введите Кол-во статей -> ");
        fgets(buffer, 75, stdin);
        buffer[strcspn(buffer, "\n")] = '\0';
        if (isNumber(buffer) != -1) key = isNumber(buffer);
        else {
            printf("\nВведено некорректное число в поле 'кол-
во статей'! Повторите ввод снова\n");
            continue;

```

```

    }
    flag = 0;
}
printTableHeader();
for (; temp != NULL; temp = temp->next) {
    if (temp->info.articles == key) {
        printNode(temp);
        count++;
    }
}
printf("Найдено %d записей с Кол-вом статей '%d'", count,
key);

return;

case 6:
    while (flag) {
        printf("Введите Кол-во цитирований -> ");
        fgets(buffer, 75, stdin);
        buffer[strcspn(buffer, "\n")] = '\0';
        if (isNumber(buffer) != -1) key = isNumber(buffer);
        else {
            printf("\nВведено некорректное число в поле 'кол-
во цитирований'! Повторите ввод снова\n");
            continue;
        }
        flag = 0;
    }
    printTableHeader();
    for (; temp != NULL; temp = temp->next) {
        if (temp->info.quotes == key) {
            printNode(temp);
            count++;
        }
    }
}

```

```

        printf("Найдено %d записей с Кол-вом цитирований '%d'",
count, key);
        return;

    case 7:
        while (flag) {
            printf("Введите Индекс Хирша -> ");
            fgets(buffer, 75, stdin);
            buffer[strcspn(buffer, "\n")] = '\0';
            if (isNumber(buffer) != -1) key = isNumber(buffer);
            else {
                printf("\nВведено некорректное число в поле
'индекс Хирша'! Повторите ввод снова\n");
                continue;
            }
            flag = 0;
        }
        printTableHeader();
        for (; temp != NULL; temp = temp->next) {
            if (temp->info.hirshIndex == key) {
                printNode(temp);
                count++;
            }
        }
        printf("Найдено %d записей с Индексом Хирша '%d'", count,
key);

        return;

    default:
        printf("Ошибка выбора!\n");
        break;
}

printf("\nНажмите любую клавишу для возврата в меню...");
getch();

```

```

}

/// @brief Функция сравнения двух элементов по выбранному полю
(вспомогательная функция для sortTable())
int compareScientist(const struct scientist *a, const struct scientist
*b, const int choice) {
    switch (choice) {
        case '1': return a->id - b->id;
        case '2': return strcmp(a->name, b->name);
        case '3': return strcmp(a->area, b->area);
        case '4': return strcmp(a->degree, b->degree);
        case '5': return a->articles - b->articles;
        case '6': return a->quotes - b->quotes;
        case '7': return a->hirshIndex - b->hirshIndex;
        default: return 0;
    }
}

/// @brief Функция сортировки таблицы по указанному полю
/// @param choice выбранное для сортировки поле
/// @param mode порядок сортировки (`1` - по убыванию | `0` - по
возрастанию)
struct list *sortTable(struct list *head, const int choice, const int
mode) {
    int swapped;
    struct list *temp, *last = NULL;
    struct scientist data;

    while (swapped) {
        swapped = 0;
        temp = head;

        for (; temp->next != last; temp = temp->next) {
            switch (mode) {
                case 1: // по убыванию

```

```

        if (compareScientist(&temp->info, &temp->next-
>info, choice) < 0) {
            data = temp->info;
            temp->info = temp->next->info;
            temp->next->info = data;
            swapped = 1;
        }
        break;

    case 0: // по возрастанию
        if (compareScientist(&temp->info, &temp->next-
>info, choice) > 0) {
            data = temp->info;
            temp->info = temp->next->info;
            temp->next->info = data;
            swapped = 1;
        }
        break;
    }
    last = temp;
}

return head;
}

/// @brief функция удаления дубликатов из текстового файла
/// @param filename имя файла
void removeDuplicatesFromFile(const char *filename) {
    FILE *f = fopen(filename, "rt");
    if (f == NULL) {
        printf("Файл %s не найден!\n", filename);
        return;
    }
}

```

```

char lines[2000][256];
int ids[2000];
int lineCount = 0;
char buffer[256];

while (fgets(buffer, sizeof(buffer), f)) {
    buffer[strcspn(buffer, "\n")] = '\0';

    int id = -1;
    if (sscanf(buffer, "%d", &id) == 1) {
        int duplicate = 0;
        for (int i = 0; i < lineCount; i++) {
            if (ids[i] == id) {
                duplicate = 1;
                break;
            }
        }
        if (!duplicate) {
            ids[lineCount] = id;
            strcpy(lines[lineCount], buffer);
            lineCount++;
        }
    } else {
        ids[lineCount] = -1;
        strcpy(lines[lineCount], buffer);
        lineCount++;
    }
}
fclose(f);

f = fopen(filename, "wt");
if (f == NULL) {
    printf("\033[31mОшибка записи в файл %s\n\033[0m", filename);
    return;
}

```

```

    for (int i = 0; i < lineCount; i++) {
        fprintf(f, "%s\n", lines[i]);
    }
    fclose(f);
}

/// @brief функция обработки согласно варианту (вывести по 5 уч. с
наибольшим индексом хирша и кол-вом цитирований для каждой научной
области)
/// @param head указатель на первый элемент
void top5ByArea(struct list *head) {
    if (head == NULL) {
        printf("Список пуст!\n");
        return;
    }

    system("cls");

    FILE *file = fopen("10results.txt", "wt");

    struct list *temp = head;

    while (temp != NULL) {
        char currentArea[26];
        strcpy(currentArea, temp->info.area);

        int processed = 0;
        struct list *prev = head;
        while (prev != temp) {
            if (strcmp(prev->info.area, currentArea) == 0) {
                processed = 1;
                break;
            }
            prev = prev->next;
        }
    }
}

```



```

if (!processed) {
    struct list *areaList = NULL;
    struct list *scan = head;

    while (scan != NULL) {
        if (strcmp(scan->info.area, currentArea) == 0) {
            struct list *newNode = (struct
list*)malloc(sizeof(struct list));
            newNode->info = scan->info;
            newNode->next = areaList;
            newNode->prev = NULL;
            if (areaList) areaList->prev = newNode;
            areaList = newNode;
        }
        scan = scan->next;
    }

    areaList = sortTable(areaList, '7', 0);

    printf("\n\n===== %s =====\n", currentArea);

    printf("\nТоп-5 по Хиршу:\n");

    struct list *show = areaList;
    printTableHeader();
    for (int i = 0; i < 5 && show != NULL; i++, show = show-
>next) {
        printNode(show);
        writeToTextFile(file, show);
    }

    areaList = sortTable(areaList, '6', 0);

    printf("\nТоп-5 по цитированиям:\n");

```

```

        show = areaList;
        printTableHeader();
        for (int i = 0; i < 5 && show != NULL; i++, show = show-
>next) {
            printNode(show);
            writeToTextFile(file, show);
        }

        deleteNode(&areaList, 0);
    }
    temp = temp->next;
}

fclose(file);
removeDuplicatesFromFile("10results.txt");

printf("\n\033[32mРезультаты в файле '10results.txt'\n");
printf("\033[0mНажмите любую клавишу для возвращения в меню...");
getch();
}

/// @brief Проверка каптчи для критических действий
/// @param action описание действия (выводится пользователю)
/// @return `1` - если проверка пройдена, `0` - если не пройдена или
отменено
int captcha(const char *action) {
    int a = rand() % 50 + 1;
    int b = rand() % 50 + 1;
    int op = rand() % 2; // 0 - сложение | 1 - вычитание
    int correctAnswer, answer;
    char input[10];

    system("cls");

```

```

if (op == 0) {
    correctAnswer = a + b;
    printf("\nПодтверждение действия: %s\n", action);
    printf("    Для продолжения решите пример: %d + %d = ", a, b);
} else {
    if (a < b) {
        int t = a;
        a = b;
        b = t;
    }
    correctAnswer = a - b;
    printf("\nПодтверждение действия: %s\n", action);
    printf("    Для продолжения решите пример: %d - %d = ", a, b);
}

fgets(input, 10, stdin);
input[strcspn(input, "\n")] = '\0';

while (1) {
    if (isNumber(input) != -1) {
        answer = isNumber(input);
        break;
    }
    else {
        printf("Введено некорректное число! Повторите ввод.\n");
        if (op == 0) {
            printf("\nПодтверждение действия: %s\n", action);
            printf("    Для продолжения решите пример: %d + %d = ",
a, b);
        } else {
            printf("\nПодтверждение действия: %s\n", action);
            printf("    Для продолжения решите пример: %d - %d = ",
a, b);
        }
    }
}

```

```

    }

    if (answer == correctAnswer) {
        printf("\033[32mПроверка пройдена. Выполняем...\033[0m\n\n");
        Sleep(2000);
        return 1;
    } else {
        printf("\033[31mНеверный ответ. Действие
отменено.\033[0m\n\n");
        Sleep(2000);
        return 0;
    }
}

/// @brief вспомогательная функция для перерасчёта длины кириллических
(utf8) символов
/// @param s строка для расчёта
/// @return длину строки с учётом разности байт
int utf8_strlen(const char *s) {
    int len = 0;
    while (*s) {
        if ((*s & 0xC0) != 0x80) len++;
        s++;
    }
    return len;
}

/// @brief Универсальная функция для отображения меню и получения
выбора пункта.
/// @param title Заголовок меню (выводится сверху)
/// @param items Массив строк с пунктами меню
/// @param itemCount Количество пунктов
/// @return Индекс выбранного пункта (0..itemCount-1) или -1, если
нажата ESC
int showMenu(const char *title, const char *items[], int itemCount) {

```

```

int current = 0;
int key;
const int width = 98;    // внутренняя ширина (между вертикальными
чертами)

while (1) {
    system("cls");

    // Верхняя граница
    printf("┌");
    for (int i = 0; i < width; i++) printf("=");
    printf("┐\n");

    // Строка заголовка с центрированием по реальной (видимой)
длине
    int visibleLen = utf8_strlen(title);
    int leftPad = (width - visibleLen) / 2;
    int rightPad = width - visibleLen - leftPad;

    printf("│");
    for (int i = 0; i < leftPad; i++) printf(" ");
    printf("%s", title);
    for (int i = 0; i < rightPad; i++) printf(" ");
    printf("│\n");

    // Нижняя граница
    printf("└");
    for (int i = 0; i < width; i++) printf("=");
    printf("┘\n");

    // Пункты меню
    for (int i = 0; i < itemCount; i++) {
        printf("  %s %s\n", (i == current) ? "\033[35m▶" : " ",
items[i]);
        printf("\033[0m");
    }
}

```

```

    }

    printf("\nИспользуйте ↑ ↓ (стрелки вверх/вниз), Enter - выбор,
    ESC - выход.\n");

    key = getch();
    if (key == 224) {
        key = getch();
        if (key == UP) current = (current - 1 + itemCount) %
itemCount;

        else if (key == DOWN) current = (current + 1) % itemCount;
    } else if (key == ENTER) {
        return current;
    } else if (key == ESC) {
        return -1;
    }
}

}

/// @brief Ввод целого неотрицательного числа с консоли с проверкой.
/// @param prompt Приглашение к вводу
/// @return Введённое число
int inputInt(const char *prompt) {
    char buffer[25];
    int value;
    while (1) {
        printf("%s", prompt);
        fgets(buffer, sizeof(buffer), stdin);
        buffer[strcspn(buffer, "\n")] = '\0';
        if (isNumber(buffer) != -1) {
            value = isNumber(buffer);
            break;
        }

        printf("\033[31mОшибка: введите целое неотрицательное
число!\033[0m\n");
    }
}

```

```

        return value;
    }

    /// @brief Обработка подменю "Сортировка"
    void handleSort(struct list *head) {
        if (!head || !head->next) {
            system("cls");
            printf("Список не нуждается в сортировке или пуст.\nНажмите любую клавишу...");
            getch();
            return;
        }
        const char *sortItems[] = {
            "По ID", "По ФИО", "По учёной степени", "По области наук",
            "По числу статей", "По числу цитирований", "По индексу Хирша",
            "Вернуться в меню"
        };
        int choice = showMenu("СОРТИРОВКА ТАБЛИЦЫ", sortItems, 8);
        if (choice == -1 || choice == 7) return; // ESC или "Вернуться"

        const char *orderItems[] = { "По возрастанию", "По убыванию",
            "Вернуться" };
        int order = showMenu("ВЫБЕРИТЕ ПОРЯДОК СОРТИРОВКИ", orderItems,
            3);
        if (order == -1 || order == 2) return;

        char sortChar = '1' + choice; // '1'..'7'
        sortTable(head, sortChar, order); // order: 0 - возр., 1 - убыв.
        printf("\nСортировка выполнена (%s). \nНажмите любую клавишу...",
            order == 0 ? "по возрастанию" : "по убыванию");
        getch();
    }

    /// @brief Обработка подменю "Поиск"
    void handleFind(struct list *head) {

```

```

    if (!head) {
        printf("Таблица пуста!\n");
        getch();
        return;
    }
    const char *findItems[] = {
        "По ID", "По ФИО", "По учёной степени", "По области наук",
        "По числу статей", "По числу цитирований", "По индексу Хирша",
"Вернуться"
    };
    int choice = showMenu("ПОИСК УЧЁНОГО", findItems, 8);
    if (choice == -1 || choice == 7) return;
    findScientist(head, choice + 1); // +1, потому что в findScientist
choice от 1 до 7
    printf("\nНажмите любую клавишу...");
    getch();
}

/// @brief Обработка подменю "Экспорт"
void handleExport(struct list *head, int *savedFlag) {
    if (!head) {
        printf("Список пуст!\n");
        getch();
        return;
    }
    const char *expItems[] = { "В текстовый файл (.txt)", "В бинарный
файл (.bin)", "Вернуться" };
    int choice = showMenu("ЭКСПОРТ ТАБЛИЦЫ", expItems, 3);
    if (choice == -1 || choice == 2) return;
    if (choice == 0) exportToTxt(head);
    else exportToBin(head);
    *savedFlag = 1;
}

/// @brief Обработка подменю "Импорт"

```



```

void handleImport(struct list **head, int *savedFlag) {
    if (*head && *savedFlag == 0) {
        if (!captcha("импорт таблицы с потерей несохранённых данных"))
            return;
        deleteNode(head, 0);
    }
    const char *impItems[] = { "Из текстового файла (.txt)", "Из
бинарного файла (.bin)", "Вернуться" };
    int choice = showMenu("ИМПОРТ ТАБЛИЦЫ", impItems, 3);
    if (choice == -1 || choice == 2) return;
    if (choice == 0) *head = importFromTxt(*head);
    else *head = importFromBin(*head);
    *savedFlag = 0;
}

/// @brief Обработка подменю "Добавление записи"
void handleAdd(struct list **head, int *savedFlag) {
    if (!*head) {
        printf("Список пуст!\nРекомендуемое действие: 1.\nНажмите
любую клавишу...");
        getch();
        return;
    }
    const char *addItems[] = { "Добавить в начало", "Добавить в
конец", "Вернуться" };
    int choice = showMenu("ДОБАВЛЕНИЕ ЗАПИСИ", addItems, 3);
    if (choice == -1 || choice == 2) return;
    struct scientist newSci = readData();
    if (choice == 0) *head = addFirst(*head, newSci);
    else *head = addLast(*head, newSci);
    *savedFlag = 0;
}

```